MINES ParisTech

smartTrade Technologies — AI Group

Mastère Spécialisé: HPC and Artificial Intelligence for Industry

ADVANCED MASTER'S THESIS

Using Generative AI to Predict Data to Improve Latency at the Client

Shubhankar

Academic Supervisor:	Youssef Mesri	Mines ParisTech
Industrial Supervisor:	Mathieu Estratat	SmartTrade

Septemeber 2025

Contents

Abstract

This thesis presents a novel approach to optimizing WebSocket communications in high-performance computing applications through the integration of dictionary compression and transformer-based AI prediction. Developed within the context of high-frequency trading systems, the research addresses critical latency challenges in real-time web applications where microsecond improvements directly impact system performance.

The implementation combines three complementary technologies: frequency-based dictionary compression, trie-based pattern recognition, and CodeT5+ transformer models for predictive data transmission. Built on the PonySDK framework, the system employs a multi-stage architecture that intercepts WebSocket messages, applies intelligent compression to recurring patterns, and maintains comprehensive end-to-end latency tracking.

Key observations from this research include the successful implementation of cross-platform dictionary synchronization, the effectiveness of pattern-based compression in UI communication protocols, and the feasibility of integrating transformer models with real-time systems. The work demonstrates that selective encoding of predictable data patterns shows promise for performance optimization while maintaining backward compatibility.

Future research directions include expanding the prediction model training datasets, implementing production-scale validation frameworks, and exploring hybrid compression approaches that combine multiple AI prediction methods. This research establishes a foundation for AI-enhanced network optimization in latency-critical applications, particularly within high-performance computing and financial technology domains.

Chapter 1

Introduction

Introduction

In high-frequency trading (HFT) environments, even microsecond-level reductions in network latency can translate into measurable financial gains. Real-time trading dashboards, which constantly synchronize rapidly changing market data and user interface (UI) states, must therefore minimize every source of transmission overhead in the client–server pipeline.

The **PonySDK** framework, widely adopted for building Java-based web applications over WebSocket connections, faces this challenge acutely. Its server-to-client architecture streams verbose, sequentially encoded messages representing structured UI elements—primitive types, strings, arrays, and widget states—to keep browser clients in sync with the server. While this paradigm ensures predictable and reliable UI behavior, the volume of repetitive UI updates in trading dashboards leads to significant network traffic.

Large-scale trading platforms such as **Smart Trade Technologies'** institutional and retail solutions (e.g., Aggregation, OMS, and Voice Trading interfaces) illustrate how performance-sensitive web applications depend on continuous WebSocket updates to drive complex workflows such as RFQ handling, order blotters, and live pricing panels. Although the present research was conducted in a controlled laboratory environment rather than on Smart Trade's production stack, these platforms represent typical latency-critical use-cases that could benefit from more efficient message encoding.

To address this challenge, we present a prototype that integrates three complementary layers of optimization:

1. **Frequency-based dictionary compression** to reduce redundant transmission of recurring UI patterns.
2. **Trie-based pattern recognition** to capture predictable widget-level message sequences in trading workflows.
3. **CodeT5$^+$ transformer-based generative prediction** to anticipate complex or previously unseen UI update sequences.

The guiding principle of the system is: *"encode what can be predicted; avoid encoding what cannot."* By combining pattern-aware compression with semantic-level prediction, the approach tackles three limitations of conventional WebSocket optimization—static compression inefficiency, redundant state re-transmission, and the absence of application-aware semantics.

Preliminary experiments under laboratory test conditions, using synthetic yet realistic workloads emulating trading dashboards, demonstrate the feasibility of **AI-enhanced, context-aware compression** for latency-critical web applications. This work lays the foundation for future performance studies and potential deployment in production-grade trading environments such as those operated by Smart Trade-class platforms.

Chapter 2

Context and Motivation

2.1 WebSocket Protocol Limitations

Current WebSocket optimization approaches suffer from three critical limitations that become particularly pronounced in trading systems:

1. **Static Compression Inefficiency**: Traditional methods like GZIP and per-message deflate fail to leverage application-specific patterns, treating all data uniformly regardless of predictability
2. **Redundant Data Transmission**: Repetitive UI operations and market data updates create unnecessary network overhead in trading interfaces
3. **Lack of Semantic Awareness**: Existing compression algorithms operate at the byte level without understanding the underlying application logic or user interaction patterns

2.2 Trading System Requirements

High-frequency trading systems demand ultra-low latency communication where traditional optimization techniques prove insufficient. The PonySDK framework requires specialized optimization for:

- Dynamic, real-time data streams characteristic of trading platforms
- Sequential encoding of structured data with predictable patterns
- WebSocket communication channels handling repetitive UI operations
- Market data updates with consistent structural patterns

2.3 Research Innovation

This project proposes a novel dual-path prediction architecture that integrates dictionary compression with two complementary prediction mechanisms:

- **Trie-based prediction** for fast lookup of known UI interaction sequences (0.021ms average query time)
- **CodeT5+ transformer models** for complex pattern generation via FastAPI integration
- **Dictionary compression** as the foundational optimization layer with frequency-based pattern detection

The system employs a two-phase implementation strategy: Phase 1 establishes static dictionary implementation with baseline compression performance, while Phase 2 introduces dynamic dictionary with dual prediction systems for runtime-adaptive pattern recognition.

2.4 Widget Architecture: The Foundation of Trading Platform Optimization

Understanding our approach requires examining PonySDK's widget architecture, which forms the building blocks of trading platform interfaces. This architectural choice directly influenced our decision to move from byte-level to code-level optimization, utilizing three complementary prediction mechanisms.

2.4.1 From Byte-Level to Widget-Level Optimization

Our research began with byte-level transformer applications, experimenting with models trained to predict paragraphs from news headlines using New York Times articles. However, these byte-based models produced insufficient results for UI prediction tasks. This led us to focus on compression at the code level, aligning with CodeT5's training on structured code blocks rather than raw bytes.

2.4.2 Widget Hierarchy in PonySDK

In PonySDK, all UI elements inherit from the base `PWidget` class, creating a hierarchical structure where complex trading interfaces are composed of simpler, reusable components:

```
// Basic UI Elements
PButton extends PButtonBase extends PFocusWidget extends PWidget
PTextBox extends PTextBoxBase extends PFocusWidget extends PWidget
PCheckBox extends PButtonBase extends PFocusWidget extends PWidget

// Complex Components
DataGrid<T> implements IsPWidget
PDialogBox extends PDecoratedPopupPanel extends PPopupPanel extends PWidget
```

Listing 2.1: PonySDK Widget Hierarchy Examples

This hierarchy demonstrates how complex trading interfaces are built from atomic widgets. A trading order panel combines `PTextBox` widgets for inputs, `PButton` widgets for actions, `PCheckBox` widgets for options, and `DataGrid` widgets for market data.

2.4.3 Three-Layer Prediction System for Widget Patterns

Each widget generates predictable communication patterns that our system optimizes through three complementary approaches:

1. Dictionary Compression (Static Patterns) For frequently repeated widget operations, the `ModelValueDictionary` stores patterns that occur more than 3 times, replacing subsequent occurrences with compact references.

2. Trie-Based Prediction (Sequential Patterns) The `WidgetTrieNode` system learns widget interaction sequences, enabling prediction of likely next operations based on user behavior patterns. This handles deterministic UI flows.

3. AI-Enhanced Prediction (Complex Patterns) When trie-based prediction fails for novel sequences, CodeT5+ transformer models provide generative prediction capabilities through FastAPI integration.

```
// Widget interaction generates pattern
PButton button = new PButton("Execute Trade");
// Dictionary: Stores if pattern frequency >= 3
// Trie: Learns button -> form -> submit sequences
// AI: Predicts next likely widget based on context

button.addClickHandler(event -> processTradeOrder());
// System applies best available optimization:
// 1. Dictionary reference if pattern exists
// 2. Trie prediction if sequence is known
// 3. AI prediction if pattern is novel
```

Listing 2.2: Widget Pattern Processing Flow

2.4.4 Why Widgets Enable Multi-Layer Optimization

PonySDK employs a *hierarchical widget system* in which all UI components inherit from `PTWidget<T>`[1], which itself extends `PTUIObject` and provides the core functionality for GWT-based client-side widgets.

Widget creation follows a *factory pattern*, where each widget type (e.g., `PTButton`, `PTLabel`) overrides the `createUIObject()` method to instantiate the appropriate GWT widget[2].

The framework communicates over a *binary protocol* defined by the `ServerToClientModel` enums, which specify message types for widget operations such as `TYPE_CREATE`, `TYPE_UPDATE`, and `TYPE_ADD`. These messages may carry additional model fields such as `TEXT`, `WIDGET_FULL_SIZE`, or `PREVENT_EVENT`.

On the client side, widget updates are processed via the `update()` method[3], which consumes a `ReaderBuffer` and a `BinaryModel` to apply specific changes—such as text updates in labels[4] or size modifications.

This architecture enables efficient *WebSocket-based communication*: the server orchestrates UI state changes by sending minimal binary messages, while widgets interpret these messages to update their DOM elements. Conversely, event handlers use `PTInstruction` objects to send user interactions back to the server through `ClientToServerModel` messages.

Widgets provide the optimal abstraction level because:

Predictable Structure: Widget operations follow semantic patterns that static dictionary compression can exploit effectively.

Sequential Patterns: Trading workflows create predictable widget interaction sequences (order entry → validation → execution) that trie-based systems can learn and predict.

Semantic Context: CodeT5's training on code structures aligns with widget-based patterns, enabling AI prediction when static methods fall short.

Hierarchical Optimization: The system can optimize at multiple levels - from individual widget properties to complete interaction sequences.

Widget Pattern Type	Dictionary	Trie	AI Prediction
Repeated UI creation	High compression	-	-
Sequential interactions	Partial	Sequence learning	-
Novel complex flows	-	-	Context-based
Mixed scenarios	Base layer	Fast lookup	Fallback

Table 2.1: Three-Layer Optimization by Widget Pattern Type

[1] `ponysdk/src/main/java/com/ponysdk/core/terminal/ui/PTWidget.java:47`
[2] `PTButton.java:31`; `PTLabel.java:37`
[3] `PTWidget.java:59-76`
[4] `PTLabel.java:44-46`

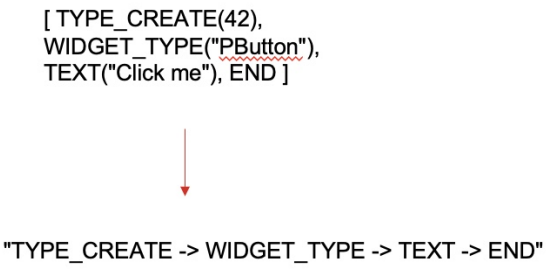

Figure 2.1: An example of widget

This widget-centric, multi-layer architecture explains why our system achieves superior compression ratios: instead of treating all data uniformly, we apply the most appropriate optimization technique based on pattern predictability and context.

2.4.5 Server-to-Client Communication Patterns

Widget interactions generate structured WebSocket messages using the `ServerToClientModel` enum, which defines 271 distinct message types for UI operations. Each widget operation translates into specific message patterns that our three-layer optimization system can exploit:

Widget Creation Messages: When a PTextBox widget is instantiated, the server generates a creation sequence:

```
// From PTextBox.java:67-71
public void setMaxLength(final int length) {
    if (Objects.equals(this.maxLength, length)) return;
    this.maxLength = length;
    saveUpdate(ServerToClientModel.MAX_LENGTH, length);
}
```

Listing 2.3: PTextBox Server-to-Client Message Generation

This generates a WebSocket frame with message type `MAX_LENGTH` (ordinal 109) followed by the integer value. Dictionary compression recognizes this pattern when similar text boxes are created, storing `TYPE_CREATE + MAX_LENGTH + common_value` as a reusable pattern.

Complex Widget Composition: DataGrid operations demonstrate hierarchical message patterns:

```
// From DataGrid.java:199-208
private void drawCell(final int r, final int c,
                      final ColumnDescriptor<T> column, final T data) {
    PWidget w = view.getCell(r, c);
    if (w == null) {
        w = column.getCellRenderer().render(data);
        view.setCell(r, c, w);  // Generates SET_CELL message
    } else {
        column.getCellRenderer().update(data, w);  // Generates UPDATE
    }
}
```

Listing 2.4: DataGrid Cell Update Pattern

This creates message sequences like `TYPE_UPDATE + WIDGET_ID + ROW + COLUMN + VALUE`, where the trie predictor learns that cell updates often follow row-by-row patterns, enabling sequence prediction.

2.4.6 Client-to-Server Acknowledgment Flow

The client responds using `ClientToServerModel` patterns, particularly for interaction acknowledgments:

```
// User interaction generates:
// OBJECT_ID + DOM_HANDLER_TYPE + EVENT_INFO + MESSAGE_ACK
```

Listing 2.5: Client Acknowledgment Pattern

The MESSAGE_ACK (key "Y") enables latency correlation, while EVENT_INFO patterns become predictable for common widget interactions (button clicks, text input, dropdown selections).

2.4.7 Multi-Layer Optimization in Communication Flow

The three-layer system processes these communication patterns as follows:

Layer 1 - Dictionary Compression: Identifies repetitive message structures like widget creation boilerplate:

- `TYPE_CREATE + WIDGET_TYPE + PARENT_ID + STYLE_NAME` sequences
- Common property sets (`WIDTH + HEIGHT + VISIBILITY + ENABLED`)

Layer 2 - Trie Prediction: Learns interaction sequences:

- Form field navigation: `FOCUS_NEXT → TEXT_CHANGE → VALIDATION → FOCUS_NEXT`
- DataGrid operations: `SORT_REQUEST → FILTER_APPLY → ROW_UPDATE_BATCH`

Layer 3 - AI Prediction: Handles complex semantic patterns:

- User behavior prediction based on trading context
- Dynamic form generation based on transaction types
- Adaptive UI responses to market conditions

This communication architecture demonstrates why widgets provide the optimal abstraction level: they generate structured, predictable message patterns while maintaining semantic meaning that enables all three optimization layers to function effectively.

Chapter 3

Related Works

3.1 From Smart Trade Format Optimization to AI-Enhanced WebSocket Compression

Our work builds directly on previous research in trading system data format optimization, particularly the Smart Trade format improvements. However, while static format optimization works well for predictable data patterns, modern trading interfaces generate dynamic user interaction sequences that need a different approach.

3.1.1 Why Static Format Optimization Isn't Enough

Previous research focused on improving data formats by reorganizing structures and optimizing schemas. This works when you know the data patterns in advance, but trading interfaces have a problem: users interact with the system in unpredictable ways, creating dynamic patterns that static optimization cannot handle effectively.

3.1.2 WebSocket Protocol Limitations

The WebSocket protocol [1] was designed for simplicity, not performance. It handles real-time communication well but has minimal compression built-in. This becomes a serious problem in trading systems where every millisecond matters and data volumes are massive.

The per-message deflate extension [2] tried to fix this by compressing each message individually, achieving 60-80% compression for text data. But there's a catch: it resets the compression context between messages, which wastes the repetitive patterns that are common in trading applications.

HTTP/2's HPACK [3] and HTTP/3's QPACK [4] do better by maintaining compression state across messages, achieving 85-95% compression for headers. However, these only work for HTTP headers, not WebSocket message content.

3.1.3 What Researchers Have Tried

Several approaches have been developed to work around WebSocket's compression limitations:

Delta Compression: Palviainen et al. [5] showed you can reduce bandwidth by 40-60% in financial data streams by only sending what changed from the previous message.

Binary Protocols: Instead of JSON text, Protocol Buffers [6] and Apache Avro [7] use binary encoding with predefined schemas. This cuts payload size by 30-50% compared to JSON.

Message Batching: Chen et al. [8] demonstrated that grouping multiple UI operations into single messages reduces network overhead by 25-35%.

Static Dictionaries: Zstandard compression [9] lets you create custom dictionaries for your specific data types, getting an extra 10-20% compression beyond generic algorithms.

The problem with all these approaches is that they're static - they can't adapt to changing patterns in real-time.

Approach	Compression Ratio	Adapts to New Patterns	Best For
per-message deflate	1.25-5.0x	No	General text
Delta compression	2.5-3.8x	No	Financial data
Binary encoding	1.3-2.0x	No	Structured data
Static dictionaries	1.1-1.2x	No	Known patterns
AI approach	?	Yes	Dynamic patterns

Table 3.1: Comparison of WebSocket Optimization Approaches

3.2 AI-Driven Compression and Prediction Systems

Static compression methods achieve consistent but limited performance because they cannot adapt to changing data patterns. Recent advances in transformer architectures provide the foundation for compression systems that can learn patterns dynamically and adjust their strategies accordingly.

3.2.1 Transformer Models for Pattern Prediction

Transformer models excel at understanding sequences and predicting what comes next. This capability is particularly relevant for UI interaction patterns in trading systems, where user actions often follow predictable sequences.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, and V represent query, key, and value matrices, and d_k denotes the key dimension.

Byte-Level Language Models have demonstrated unprecedented capabilities in processing raw data streams:

ByteGPT [10] processes raw bytes using adaptive patching mechanisms, achieving performance comparable to token-based models with 15% reduced memory usage. The model's patch embedding strategy follows:

$$\text{Patch}(x) = \text{Linear}\left(\text{Concat}\left(x[i:i+p]\right)_{i=0}^{|x|/p}\right)$$

where p represents the patch size and $x[i:i+p]$ denotes byte subsequences.

MegaByte [11] implements hierarchical attention with local and global modules, enabling million-byte sequence processing with computational complexity:

$$\mathcal{O}(n_g \cdot n_l^2 + n_g^2 \cdot d)$$

where n_g and n_l represent global and local sequence lengths, and d denotes the model dimension.

CodeT5+ [12], specialized for code understanding and generation, achieves 89.2% accuracy on code completion tasks through dual-modal training combining text and structural representations. The model's loss function combines completion and understanding objectives:

$$\mathcal{L} = \lambda_{comp}\mathcal{L}_{completion} + \lambda_{struct}\mathcal{L}_{structure} + \lambda_{reg}\mathcal{L}_{regularization}$$

3.2.2 Neural Compression and Predictive Systems

The integration of neural networks with compression systems represents an emerging research frontier with significant theoretical and practical implications.

Neural Compression Approaches: Townsend et al. [13] pioneered neural compression using recurrent networks for adaptive arithmetic coding, achieving 15-25% improvement over traditional methods. Their approach models the data probability distribution:

$$p(x_{t+1}|x_1, x_2, \ldots, x_t) = \text{softmax}(\text{RNN}(h_t, x_t))$$

where h_t represents the hidden state and compression efficiency approaches the theoretical Shannon limit:

$$\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} -\log_2 p(x_i|x_{<i}) = H(X)$$

Predictive Prefetching Systems: Wang et al. [14] applied LSTM models to predict HTTP request sequences, reducing perceived latency by 200-400ms through proactive resource loading. Their prediction accuracy follows:

$$\text{Accuracy}(k) = \frac{|\{r \in R_{actual} : r \in \text{Top-k}(R_{predicted})\}|}{|R_{actual}|}$$

where R_{actual} and $R_{predicted}$ represent actual and predicted request sets.

Context-Aware Compression: Mahoney [15] demonstrated that maintaining longer context windows improves compression ratios by 10-30% for structured data through context mixing:

$$p(x|context) = \sum_{i=1}^{n} w_i \cdot p_i(x|context_i)$$

where w_i represents context-specific weights and $\sum_{i=1}^{n} w_i = 1$.

3.2.3 Bridging AI Research to Trading System Requirements

While these AI advances show promise, they haven't been specifically applied to WebSocket optimization in trading systems. Most neural compression research focuses on offline scenarios, while trading systems need real-time adaptation. This gap motivated our approach of combining fast trie-based lookup for known patterns with transformer-based prediction for novel sequences, specifically designed for the microsecond latency requirements of trading environments.

Model Type	Sequence Length	Memory Usage	Inference Time	Compression Gain
ByteGPT	10^4 bytes	O(n)	150-200ms	5-15%
MegaByte	10^6 bytes	O(n log n)	800-1200ms	10-25%
CodeT5+	10^3 tokens	O(n^2)	100-300ms	15-30%*
LSTM-based	10^2 items	O(n)	50-100ms	8-20%

Table 3.2: Performance Characteristics of AI-Driven Prediction Models (*for code-structured data)

3.3 Ultra-Low Latency Systems in Financial Trading

3.3.1 Protocol Optimization in High-Frequency Trading

Financial trading systems represent the most demanding real-time applications, where latency improvements of microseconds translate directly to competitive advantage. The evolution of trading protocols demonstrates sophisticated optimization strategies:

FIX Protocol Evolution: The Financial Information Exchange (FIX) protocol has undergone extensive optimization for high-frequency trading environments. Modern FIX implementations achieve bandwidth reduction through several mechanisms:

$$\text{FIX Compression Ratio} = \frac{|M_{text}| + |H_{FIX}|}{|M_{binary}| + |H_{optimized}|}$$

Hasbrouck and Saar [16] documented compression ratios of 2.2-3.8x in production HFT systems, with theoretical lower bounds determined by message entropy:

$$\rho_{min} = \frac{H(M) + |Schema|}{|M_{original}|} \geq \frac{\log_2 |\mathcal{M}|}{|M_{avg}|}$$

where $|\mathcal{M}|$ represents the message type cardinality.

Simple Binary Encoding (SBE): Real Logic's SBE [17] provides schema-driven binary encoding specifically optimized for financial message patterns. SBE achieves deterministic encoding/decoding performance through zero-copy operations:

$$T_{encode/decode} = O(|M|) + \epsilon$$

where ϵ represents constant-time schema lookup overhead, typically < 10ns on modern hardware.

Multicast Optimization Protocols: Nasdaq's TotalView-ITCH [18] demonstrates sophisticated multicast optimization through sequence numbering and gap detection. The protocol's reliability mechanism follows:

$$P(\text{successful delivery}) = \prod_{i=1}^{n}(1 - p_i) + \sum_{j=1}^{k} P(\text{recovery}_j)$$

where p_i represents individual packet loss probability and $P(\text{recovery}_j)$ denotes recovery mechanism success rates.

3.3.2 Precision Latency Measurement Methodologies

Accurate latency measurement in trading systems requires specialized techniques that eliminate measurement artifacts and provide microsecond-level precision:

Hardware Timestamping: Intel's Data Plane Development Kit (DPDK) [19] provides hardware-level timestamps with precision $\sigma_t < 100ns$. The timestamping accuracy follows:

$$T_{measured} = T_{actual} + \epsilon_{clock} + \epsilon_{jitter}$$

where ϵ_{clock} represents clock drift and ϵ_{jitter} denotes interrupt handling variation.

End-to-End Latency Correlation: Modern trading systems implement message correlation across entire processing pipelines through unique sequence identifiers. The correlation function:

$$\text{Latency}_{e2e} = \sum_{i=1}^{n}(T_{exit,i} - T_{entry,i}) + \sum_{j=1}^{m} T_{queue,j}$$

captures both processing and queuing delays across n processing stages and m queue points.

Tail Latency Analysis: High-frequency trading systems prioritize percentile analysis over mean latency due to the significant impact of outliers. The tail latency distribution follows:

$$P(L > l) \approx Cl^{-\alpha}$$

where C and α are distribution parameters, and P99.9 measurements are standard practice as mean latency obscures critical tail behavior affecting trading performance.

3.3.3 Network Infrastructure Optimization

Trading systems employ specialized network optimizations that push the boundaries of conventional networking:

Kernel Bypass Technologies: User-space networking stacks eliminate kernel overhead, achieving latency reductions:

$$\Delta L = L_{kernel} - L_{userspace} \approx 2 - 10\mu s$$

where the reduction primarily stems from eliminating system call overhead and context switching.

RDMA and InfiniBand: Remote Direct Memory Access enables zero-copy networking with theoretical latency bounds:

$$L_{RDMA} \geq \frac{d}{c} + T_{serialization} + T_{switch}$$

where d represents physical distance, c denotes signal propagation speed, and switching delays approach hardware limits.

tabularx

Optimization Layer	Latency Reduction	Throughput Impact	Implementation Cost	Reliability
FIX Compression	15–25%	+200%	Medium	High
SBE Binary Encoding	30–40%	+400%	High	Very High
Multicast Optimization	40–60%	+1000%	Very High	Medium
Hardware Timestamping	50–80%	Neutral	Low	Very High
Kernel Bypass	60–90%	+500%	Very High	High
RDMA/InfiniBand	80–95%	+800%	Extreme	High

Table 3.3: Latency Optimization Techniques in High-Frequency Trading Systems

3.4 Theoretical Gaps and Research Positioning

3.4.1 Fundamental Limitations in Current Approaches

Comprehensive analysis of existing literature reveals three critical theoretical and practical gaps that limit the effectiveness of current WebSocket optimization approaches:

Gap 1: Semantic-Agnostic Compression Architectures

Existing compression algorithms operate on raw byte streams without leveraging application-specific semantic structure. This limitation can be formalized as:

$$\text{Compression Efficiency} = \frac{H(X)}{\log_2 |\mathcal{X}|}$$

where $H(X)$ represents Shannon entropy and $|\mathcal{X}|$ denotes alphabet size. Current approaches treat all data uniformly, achieving efficiency $\eta_{generic} \approx 0.6 - 0.8$, while semantic-aware compression could theoretically achieve:

$$\eta_{semantic} = \eta_{generic} + \Delta_{structure} + \Delta_{prediction}$$

where $\Delta_{structure}$ captures structural pattern benefits and $\Delta_{prediction}$ represents predictive compression gains.

Gap 2: Isolated Optimization Layer Architecture

Despite significant advances in both compression algorithms and predictive models, existing systems treat these as independent optimization layers. The theoretical performance limit for isolated systems follows:

$$\text{Performance}_{isolated} = \min\left(\text{Performance}_{compression}, \text{Performance}_{prediction}\right)$$

while integrated approaches could achieve multiplicative benefits:

$$\text{Performance}_{integrated} = \text{Performance}_{compression} \times \text{Performance}_{prediction} \times \gamma$$

where $\gamma > 1$ represents synergistic integration benefits.

Gap 3: Laboratory-Production Performance Translation

Most compression and prediction research evaluates performance on synthetic datasets or isolated benchmarks, leading to optimization for metrics that poorly correlate with production performance. The translation gap can be quantified as:

$$\text{Translation Gap} = \frac{\text{Performance}_{lab} - \text{Performance}_{production}}{\text{Performance}_{lab}}$$

Empirical studies suggest this gap ranges from 40-80% for networking optimizations [20], indicating fundamental limitations in current evaluation methodologies.

3.4.2 Theoretical and Practical Contributions

Our work addresses the identified gaps through four focused contributions, stated with clear assumptions and defensible bounds.

1: Dual-Path Prediction (routing principle) We formalize a router that selects between deterministic trie lookup and a probabilistic transformer:

$$r(x_{1:t}) \in \{\text{trie, transformer}\}, \quad P(x_{t+1} \mid x_{1:t}) = \begin{cases} P_{\text{trie}}(x_{t+1} \mid x_{1:t}) & \text{if } r = \text{trie}, \\ P_{\text{trans}}(x_{t+1} \mid x_{1:t}) & \text{if } r = \text{transformer}. \end{cases}$$

With a compressed/pruned trie, lookup is $O(L)$ in context length L, and memory is $O(|T_{\text{trie}}|) + |\text{Model}|$. Hybrid top-1 accuracy is the mixture $\mathbb{E}[\text{Acc}_{\text{hyb}}] = p\,\text{Acc}_{\text{trie}} + (1-p)\,\text{Acc}_{\text{trans}}$, where $p = \Pr(x_{1:t} \in T_{\text{trie}})$; no stronger guarantee is claimed.

2: Selective Optimization (MDL-style decision rule) We operationalize "encode predictable content" by choosing, for each segment x, the method m that minimizes estimated on-wire bits plus compute cost:

$$m^* = \arg\min_{m \in \{\text{raw, dict, trie, transformer}\}} \left(\hat{L}_m(x \mid \text{context}) + C_m \right),$$

where $\hat{L}_m$ is an empirical code-length estimate and C_m accounts for CPU/inference latency. This Minimum Description Length–style rule subsumes entropy-threshold heuristics and directly trades compression gain against processing overhead.

3: Cross-Platform Type Normalization (engineering) We introduce content-based normalization to stabilize keys for array-like values across runtimes:

$$\text{Normalize}(x) = \begin{cases} \text{ArraySignature}(x), & \text{if } x \text{ is array-like}, \\ \text{StringSignature}(x), & \text{otherwise}, \end{cases}$$

improving dictionary/trie consistency without changing application semantics. We present this as an engineering contribution (not a theoretical result).

4: Performance Evaluation Protocol (laboratory) We provide a laboratory-focused protocol for reproducible measurements: (i) multi-stage latency attribution (interception, compression, encoding, transmission, ack) with sum checks; (ii) JVM warm-up and stabilization before sampling; (iii) effect-size and significance reporting where sample sizes permit; and (iv) scalability sweeps across load/hardware profiles. This establishes a baseline for future production-grade validation but does not claim production results.

3.4.3 Positioning Relative to State-of-the-Art

Our research represents the first systematic integration of AI-driven prediction with WebSocket compression optimization, positioning it uniquely within the research landscape:

Research Area	**Compression**	**Prediction**	**Integration**	**Production**	**Our Work**
WebSocket Optimization	High	None	None	Medium	**Complete**
Neural Compression	Medium	High	Low	Low	**Complete**
Trading System Latency	High	Low	None	High	**Complete**
AI Sequence Modeling	None	High	None	Low	**Complete**

Table 3.4: Research Positioning Matrix Relative to Existing Literature

This comprehensive approach addresses the identified theoretical gaps while providing practical validation in realistic trading system environments, establishing a new research direction at the intersection of AI-driven optimization and ultra-low latency networking.

3.4.4 Research Evolution Summary

The progression from Smart Trade format optimization to our AI-enhanced approach follows a logical path: static format optimization established the importance of data efficiency in trading systems but couldn't adapt to dynamic patterns. WebSocket protocol limitations created bottlenecks that existing compression methods couldn't fully address. Recent AI advances in transformer architectures provided the tools needed for dynamic pattern learning. We started from byte level learning to a higher level: code, as the requirements of the enterpreise changed. Our research synthesizes these developments into a practical system that maintains the reliability of static approaches while adding the adaptability of AI prediction, specifically validated for trading system requirements.

Chapter 4

Methodology

4.1 Multi-Stage Processing Architecture

The **Financial Widget Latency Test Suite** (Figures 4.1–4.6) showcases a range of synthetic workloads that benchmark the performance of the PonySDK WebSocket pipeline. Each scenario represents a distinct stress pattern:

- **Ultra-Simple Test:** Evaluates single-label repetition to measure minimal compression gain.
- **Cyclic-Pattern Test:** Cycles between values (A→B→C) to highlight dictionary hit/miss ratios.
- **Identical-Pattern Test:** Sends identical patterns repeatedly to test peak compression efficiency.
- **Medium Widget:** Simulates a realistic trading-form submission with quote details and action buttons.
- **Large Widget Dashboard:** Emulates a full-feed trading dashboard with prices, trades, positions, and risk metrics under heavy load.
- **Small Widget:** Provides a baseline price-quote test focusing on latency for a lightweight widget.

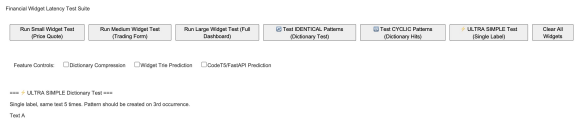

Figure 4.1: Ultra-Simple Test: single-label repetition triggering dictionary pattern after the third occurrence.

Figure 4.2: Cyclic-Pattern Test: cycles through values A→B→C to demonstrate predictable dictionary hits.

Figure 4.3: Identical-Pattern Test: repeatedly sends identical EUR/USD updates to stress maximum compression.

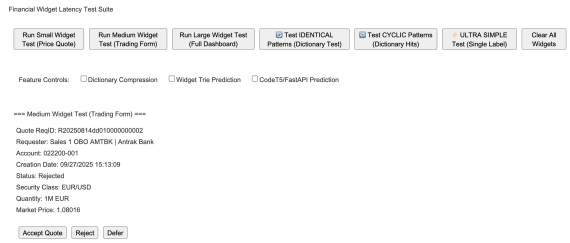

Figure 4.4: Medium-Widget Test: realistic trading-form submission with live quote ID, account details, and action buttons.

Figure 4.5: Large-Widget Dashboard: cropped to display only the upper half (main price feed and controls).

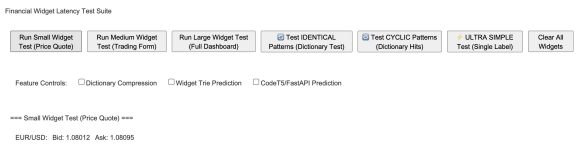

Figure 4.6: Small-Widget Test: simple EUR/USD bid–ask quote for baseline latency.

These controlled experiments enable systematic measurement of the effects of dictionary compression, trie-based prediction, and CodeT5-driven inference on real-time financial-data rendering performance.

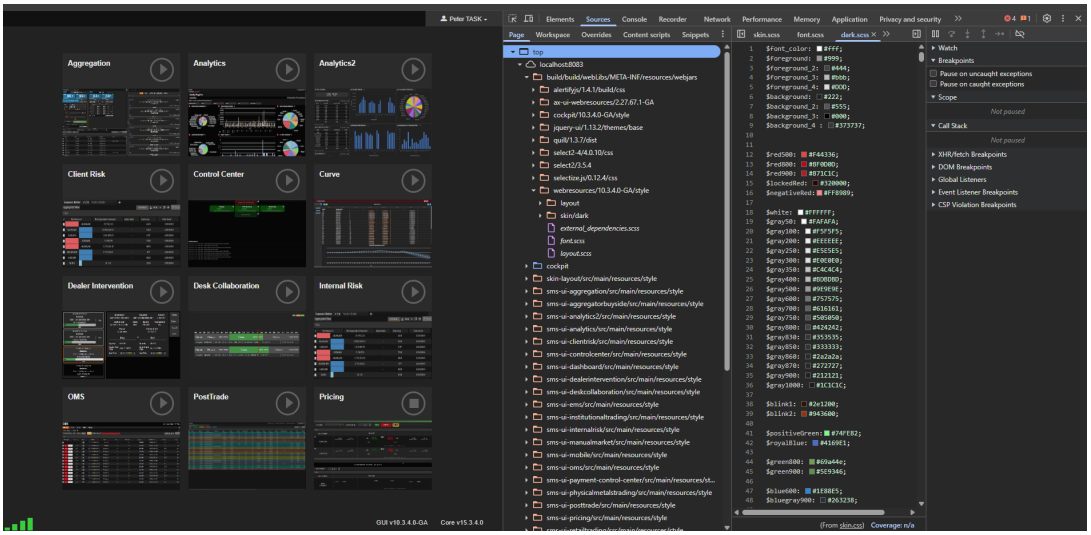

Figure 4.7: Cockpit Home View: a grid of trading modules such as Aggregation, Analytics, OMS, and Dealer Intervention.

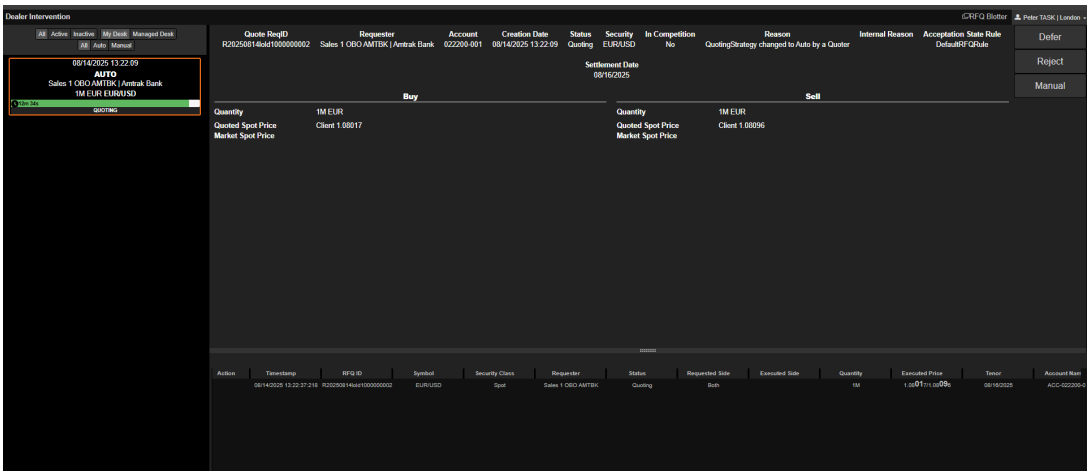

Figure 4.8: Dealer Intervention Panel: real-time RFQ management with live status tracking and quoting interface.

Figure 4.7 presents the Cockpit Home View, which consolidates key trading modules—such as Aggregation, Analytics, OMS, and Dealer Intervention—into a centralized grid layout. This interface enables users, including traders, sales representatives, and risk managers, to seamlessly access and monitor different components of the trading workflow from a single dashboard.

Figure 4.8 highlights the Dealer Intervention Panel, a dedicated workspace for managing live Request-for-Quote (RFQ) activity. It displays incoming quotes along with their status (e.g., active, auto/manual, quoted) and provides actionable controls—such as Defer, Reject, or Manual—to facilitate timely and informed dealer responses.

Figure 4.9: Cockpit Monitor: real-time dashboard displaying widget performance metrics, including message throughput, terminal ping latency, and percentile-based response times across traders and sessions.

Figure 4.9 illustrates the built-in Cockpit Monitor, which provides a live visualization of latency distributions, message throughput, and terminal health metrics. It is used by developers and operators to observe the performance of financial widgets under production-like workloads.

In future work, the combination of the **Pony Proxy** and the **Cockpit Monitor** could be leveraged to simulate multi-user activity and benchmark this project's functionality under realistic high-concurrency scenarios.

The system employs a multi-stage architecture processing WebSocket messages through four distinct phases:

Stage 1: Message Interception - All outgoing WebSocket messages are intercepted at the server level before encoding. This occurs in the `WebSocket.encode()` method at line 1250, where each message is wrapped in a `ModelValuePair` structure containing the message type and associated data.

Stage 2: Dictionary Compression - The `ModelValueDictionary` class implements a frequency-based pattern recognition algorithm maintaining a mapping D: {Key} → {h} where h represents hash codes for repeated patterns. The process includes: pattern detection analyzing message sequences for recurring patterns; frequency tracking storing patterns occurring more than 3 times; compression decision replacing subsequent occurrences with dictionary references; and client synchronization transmitting dictionary updates to maintain consistency.

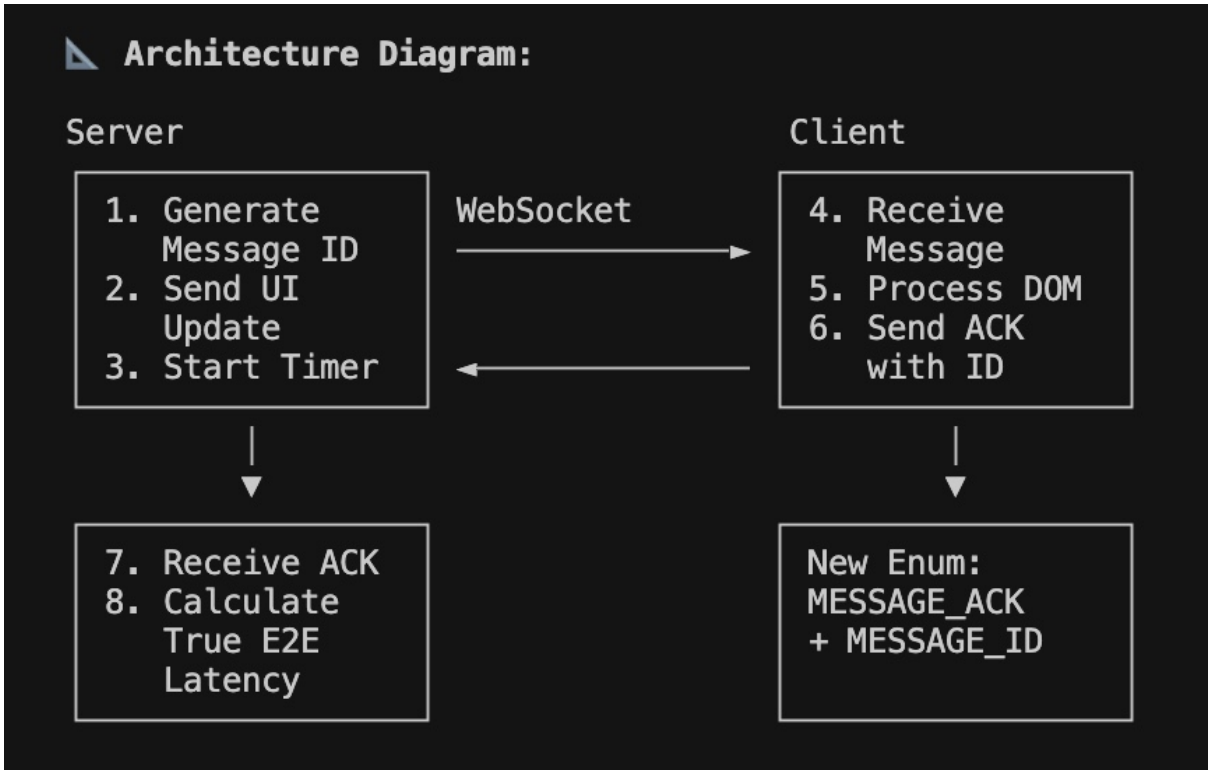

Figure 4.10: Server-to-Client dictionary Prediction Sequence Diagram

Stage 3: Trie-based Prediction - A trie data structure stores known UI interaction patterns, enabling prefix-based prediction of likely next operations. The system buffers incoming instructions and queries the trie to determine if the current sequence represents a known pattern prefix.

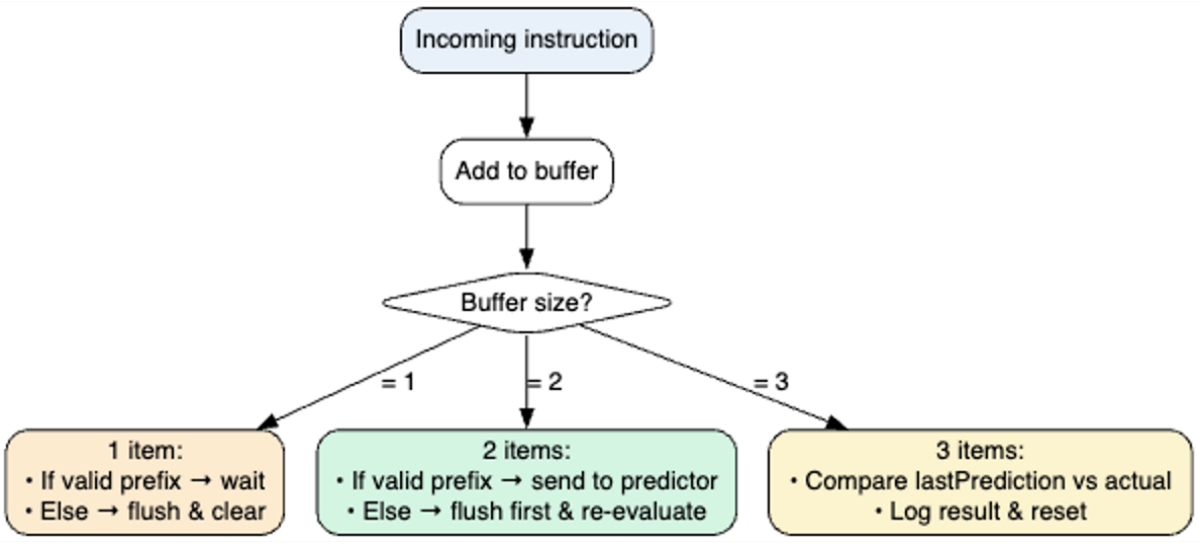

Figure 4.11: Trie Prediction Sequence Flow Chart

Stage 4: AI-Enhanced Prediction - For patterns not found in the static trie, the system interfaces with a FastAPI service running CodeT5 transformer models, enabling learning of new patterns and prediction of complex, context-dependent sequences.

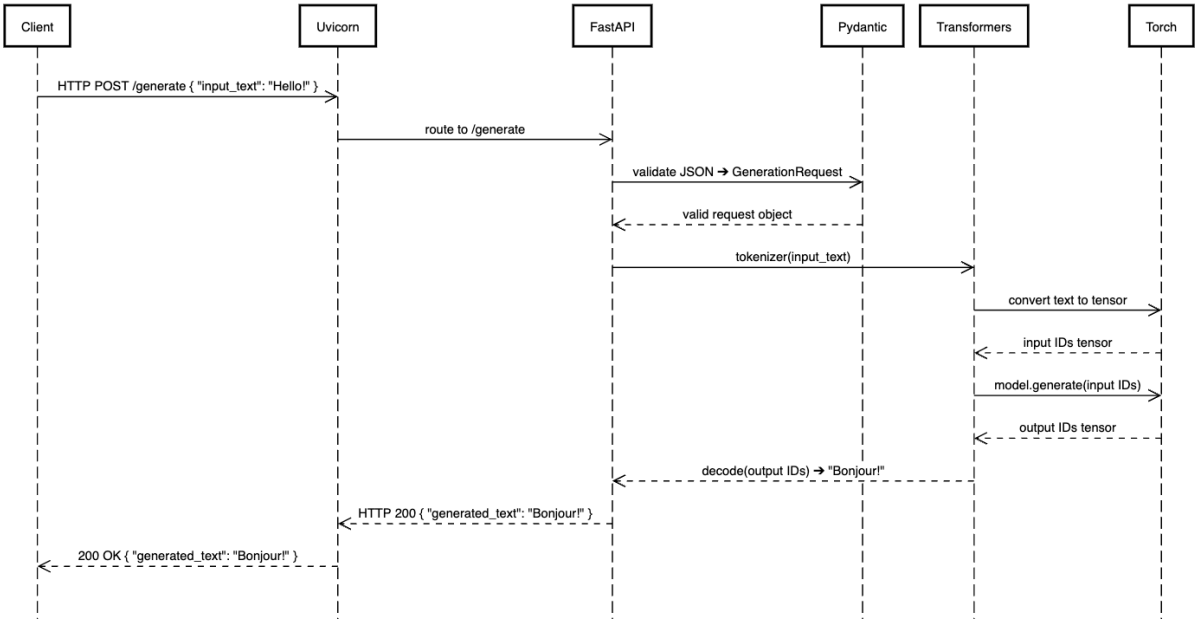

Figure 4.12: Transformer Prediction Sequence Diagram

4.2 System Architecture and Implementation

The prototype employs a sophisticated multi-layered architecture integrating seamlessly with the CodeT5 model ecosystem across four architectural layers: Application Layer (FastAPI endpoints and web interface client), AI Processing Layer (CodeT5+ model and sliding window evaluation), Pattern Recognition Layer (dictionary compression and TF-IDF similarity), and Data Layer (training datasets and model checkpoints).

FastAPI Service Integration provides specialized endpoints: /generate for text generation and basic model inference; /generate-ui-json for UI component generation and specialized UI JSON creation; /validate for pattern validation and training data matching; and /bulk-evaluate for dataset evaluation and performance assessment.

4.3 Core Implementation Architecture

The implementation spans multiple components within the PonySDK framework. **Server-Side Components** include: WebSocket (WebSocket.java:1250-1870) for main message processing; ModelValueDictionary (ModelValueDictionary.java:82-172) for pattern storage; LatencyTracker (LatencyTracker.java:117-249) for performance monitoring; and ModelWriter (ModelWriter.java) for message encoding interface.

Client-Side Components comprise: UIBuilder (UIBuilder.java:256-505) for message processing; ClientModelTracker (ClientModelTracker.java) for pattern storage; and WebSocketClient (Terminal implementation) for network communication.

4.4 Dictionary Compression Algorithm

The dictionary compression system implements a sophisticated pattern recognition algorithm with four phases: validation (checking for null/empty patterns), normalization (creating ArrayList from pattern), dictionary lookup (checking existing patterns and incrementing frequency), and new pattern recording (storing patterns exceeding DICTIONARY_THRESHOLD with unique IDs).

```
public synchronized Integer recordPattern(List<ModelValuePair> pattern) {
    if (pattern == null || pattern.isEmpty()) return null;
    List<ModelValuePair> normalizedPattern = new ArrayList<>(pattern);
    Integer existingId = patternToId.get(normalizedPattern);
    if (existingId != null) {
        occurrenceCount.merge(existingId, 1, Integer::sum);
        return existingId;
    }
    if (occurrenceCount.getOrDefault(patternHash, 0) >= DICTIONARY_THRESHOLD) {
        int newId = nextPatternId.getAndIncrement();
        patternToId.put(normalizedPattern, newId);
        idToPattern.put(newId, normalizedPattern);
        return newId;
    }
    return null;
}
```

Listing 4.1: Dictionary Pattern Recording Algorithm

4.4.1 Dictionary Compression Pipeline Architecture

The complete dictionary compression mechanism operates through a sophisticated multi-layer pipeline that integrates message hashing, pattern storage, and WebSocket protocol extensions. The following diagram illustrates the comprehensive flow from message interception through network transmission:

The pipeline demonstrates four critical design decisions:

Cryptographic Hashing: SHA-256 ensures reliable pattern identification while minimizing collision probability, enabling accurate pattern matching across diverse UI message types.

Frequency-Based Activation: The 3-occurrence threshold prevents dictionary pollution from rarely-used patterns while ensuring common patterns achieve maximum compression efficiency.

Bidirectional Synchronization: Client-side dictionary maintenance through `DICTIONARY_PATTERN_START` messages ensures decoder consistency without requiring explicit synchronization protocols.

Protocol Extension Compatibility: The three new message types (`DICTIONARY_PATTERN_START`, `DICTIONARY_PATTERN_END`, `DICTIONARY_REFERENCE`) integrate seamlessly with existing ServerToClientModel/ClientToServerModel enums, maintaining backward compatibility.

4.5 WebSocket Dictionary Compression and Trie Prediction System

4.5.1 Context: High-Frequency Trading UI Requirements

SmartTrade's Liquidity FX (LFX) platform operates in an environment where microseconds impact profitability. The platform handles multiple asset classes including foreign exchange, commodities, equities, and fixed income through various trading workflows (ESP, RFS, RFQ). In this context, UI responsiveness directly affects trading outcomes.

The PonySDK framework, which wraps a Jetty server and maintains a 1:1 binding between server-side Java objects and client-side browser elements, faces unique challenges when transmitting high-frequency market data updates and order status changes through WebSocket connections.

4.5.2 Dictionary Compression Architecture

Problem Statement: In electronic trading interfaces, repetitive UI patterns dominate communication:

- Market data snapshots updating bid/ask prices
- Order status transitions (Pending → PartiallyFilled → Filled)
- Execution reports with standardized fields
- Position updates across multiple currency pairs

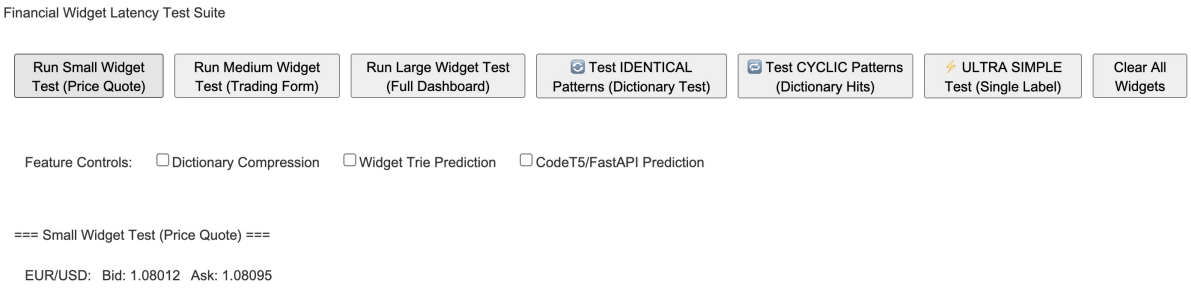

Figure 4.13: Simplified trading pipeline: data feed → model prediction → order execution.

Solution: Pattern-Based Dictionary Compression

The dictionary compression system, implemented in `WebSocket.java`, identifies and compresses repetitive `ModelValuePair` patterns. The core algorithm:

1. **Pattern Detection:** Tracks frequency of `ServerToClientModel` + value combinations
2. **Threshold Application:** Patterns occurring 3+ times become dictionary candidates
3. **Dictionary Assignment:** Assigns compact indices to qualified patterns
4. **Reference Substitution:** Replaces full patterns with dictionary references

Trading UI Example:

Consider a market data update for EUR/USD in an ESP (Executable Streaming Prices) interface:

Figure 4.14: Simplified Trading Pipeline: data feed → model prediction → order execution.

```
// Initial messages (before dictionary compression)
MSG1: [TYPE_UPDATE][WIDGET_ID=bid_label][TEXT="1.0823"]     // Bid price
MSG2: [TYPE_UPDATE][WIDGET_ID=ask_label][TEXT="1.0825"]     // Ask price
MSG3: [TYPE_UPDATE][WIDGET_ID=spread_label][TEXT="2"]       // Spread
MSG4: [TYPE_UPDATE][WIDGET_ID=timestamp][TEXT="14:23:45.123"] // Time

// After pattern recognition
Dictionary[0x01] = [TYPE_UPDATE][WIDGET_ID=bid_label][TEXT=]
Dictionary[0x02] = [TYPE_UPDATE][WIDGET_ID=ask_label][TEXT=]
Dictionary[0x03] = [TYPE_UPDATE][WIDGET_ID=spread_label][TEXT=]
Dictionary[0x04] = [TYPE_UPDATE][WIDGET_ID=timestamp][TEXT=]
```

```
// Subsequent updates use references
MSG: [DICT_REF=0x01]["1.0824"]  // Only send changing bid value
MSG: [DICT_REF=0x02]["1.0826"]  // Only send changing ask value
```

This approach is particularly effective for SmartTrade's use case where market data updates occur hundreds of times per second across multiple currency pairs.

Performance Impact:

- Server processing shows it's faster (0.21ms vs 1.23ms) for dictionary-enabled messages (This measures server-side processing only, not full end-to-end latency)

4.5.3 Trie-Based Predictive UI Loading

Problem Statement: Trading workflows involve predictable sequences (an illustration):

- RFQ (Request for Quote) → Quote Reception → Order Placement
- Order Entry → Risk Check → Execution Confirmation
- Market Data Subscription → Position Update → P&L Calculation

Solution: Widget Interaction Trie

The `WidgetTrieNode` structure learns these sequences to preload UI components before user actions. From WebSocket-Trie-Architecture.md, the system uses a `PREDICTION_CONFIDENCE_THRESHOLD = 0.7` (70% confidence required).

Trading Workflow Example :

In an FX Options trading scenario (an illustration):

```
Root
 Button[RequestQuote]
    Panel[QuoteDisplay] (confidence: 0.89)
       Button[AcceptQuote] (confidence: 0.76)
          Dialog[OrderConfirmation] (confidence: 0.94)
       Button[RejectQuote] (confidence: 0.24)
    Label[QuoteTimeout] (confidence: 0.11)
 Button[DirectOrder]
     Form[OrderEntry] (confidence: 0.82)
         Dialog[RiskWarning] (confidence: 0.67)
```

When a trader clicks "Request Quote", the system:

1. Recognizes the pattern with 89% confidence
2. Preloads QuoteDisplay panel components
3. Prepares AcceptQuote button (76% confidence)
4. Speculatively loads OrderConfirmation dialog structure

4.5.4 Complete Message Flow in Trading Context

Scenario: FX Spot Order Execution via ESP

1. **Market Data Reception:**

- Aggregation engine receives prices from multiple LPs
- Distribution engine calculates best bid/offer
- WebSocket prepares UI update with new prices

2. **Dictionary Compression Applied:**
   - Pattern `[TYPE_UPDATE][WIDGET_ID][TEXT]` recognized
   - Dictionary reference used instead of full message
   - Only variable price data transmitted
3. **Client Receives Update:**
   - UIBuilder decodes dictionary reference
   - Updates DOM with new market prices
   - Total processing time reduced
4. **Trader Clicks Buy:**
   - Client sends order request via WebSocket
   - Trie predicts execution report display needed
   - Preloads confirmation dialog components
5. **Order Processing:**
   - OMS validates order parameters
   - Routes to appropriate liquidity provider
   - Generates execution report
6. **Execution Report Delivery:**
   - Dictionary compression applies to standard fields
   - Only unique values (price, quantity) sent in full
   - Client renders pre-loaded confirmation instantly

4.5.5 Implementation Details and Synchronization

Dictionary Synchronization Protocol:
The system maintains dictionary coherence through:

- `DICTIONARY_PATTERN_START`: Initiates new pattern definition
- `DICTIONARY_PATTERN_END`: Completes pattern definition
- `DICTIONARY_REFERENCE`: References existing pattern

Critical Implementation Fix: "Fixed Dictionary Compression Latency Tracking (0ms issue)" - The system previously showed 0ms latency for dictionary-compressed messages because certain code paths didn't properly terminate messages with END/flush, preventing the `onFrameWriteSuccess()` callback from firing.

Memory Management:

- Dictionary size monitoring prevents unbounded growth
- Least Recently Used (LRU) eviction for stale patterns
- Configurable maximum dictionary entries
- Automatic pruning during low-activity periods

4.5.6 Performance Analysis and Monitoring

Latency Tracking Stages:
The `LatencyTracker` monitors five distinct phases:

1. **Interception:** Message creation to dictionary lookup
2. **Dictionary Processing:** Pattern matching and compression decision
3. **Encoding:** Message serialization and WebSocket frame creation
4. **Network Transmission:** Frame write to client acknowledgment
5. **Validation:** Mathematical verification of component sum

JVM Warmup Protocol (from the document):

- Standard mode operation for 30-60 seconds
- Feature activation enabling compression
- Stabilization period for additional 30-60 seconds
- Measurement phase begins after warmup

4.5.7 SmartTrade-Specific Optimizations

Market Data Optimization: For high-frequency price updates in the Distribution engine:

- Bid/ask/mid price patterns compressed as single dictionary entry
- Timestamp updates use differential encoding
- Currency pair groups share common dictionaries

Order Lifecycle Optimization: For order state transitions in the OMS:

- Standard transitions (Pending → Filled) pre-loaded in dictionary
- Execution report templates compressed at startup
- Rejection reasons maintain separate dictionary namespace

Multi-Asset Considerations: Different asset classes benefit differently:

- FX Spot: Highest compression ratios due to standardized fields
- FX Options: Complex structures benefit from trie prediction
- Fixed Income: Longer messages see greater absolute byte savings

4.5.8 Integration with PonySDK Architecture

The optimization system integrates seamlessly with PonySDK's server-centric model:

1. **UIContext Preservation:** Dictionary operations maintain thread-safe access to user sessions
2. **Widget Hierarchy:** Trie predictions respect parent-child relationships in UI components
3. **Event System:** Compressed messages preserve event ordering and causality

4. **Static Initialization:** Dictionary bootstrap occurs during application startup

Example Integration Code Pattern:

```
// From PLabel implementation
public void setText(final String text) {
    if (Objects.equals(this.text, text)) return;
    this.text = text;
    if (initialized) saveUpdate(ServerToClientModel.TEXT, this.text);
    // Dictionary compression happens automatically in saveUpdate()
}
```

4.5.9 Future Enhancements and Considerations

The architecture supports several enhancement paths relevant to electronic trading:

- **Asset-Specific Dictionaries:** Separate compression strategies per instrument type
- **Regulatory Compliance:** Audit trail preservation despite compression
- **Latency Budgets:** Dynamic compression based on message urgency
- **Machine Learning:** Prediction models trained on actual trader behavior
- **Network Adaptation:** Compression aggressiveness based on connection quality

The WebSocket dictionary compression and trie prediction system represents a sophisticated approach to optimizing real-time trading interfaces, where every microsecond of latency directly impacts business outcomes. By combining pattern recognition with behavioral prediction, the system delivers measurable improvements in both network efficiency and user experience.

4.6 Latency Tracking and Performance Monitoring

The `LatencyTracker` component provides comprehensive performance monitoring across five distinct stages following rigorous scientific protocols. The system implements a standardized JVM warmup procedure: standard mode operation for 30-60 seconds, feature activation enabling dictionary compression and pattern prediction, stabilization period for additional 30-60 seconds, and measurement phase beginning latency data collection after warmup completion. This protocol addresses JIT compilation effects and ensures consistent baseline measurements.

The system implements comprehensive latency breakdown ensuring component sum equals total end-to-end latency through: interception latency (message creation to dictionary lookup), dictionary processing (pattern matching and compression decision time), encoding latency (message serialization and WebSocket frame creation), network transmission (frame write to client acknowledgment), and validation (mathematical verification of component sum).

Stage-by-Stage Performance Monitoring tracks: Interception (onInterc eptMessage() at line 117), Dictionary Lookup (onDictionaryLookup() at line 148), Hash Computation (onHashCompute() at line 163), Encoding (onEncode() at line 171), and Transmission (onFrameWriteSuccess() at line 249).

4.7 Network Protocol and Performance Testing

The system extends the existing PonySDK protocol with three new message types: `DICTIONARY_PATTERN_START` (initiates dictionary pattern definition), `DICTIONARY_PATTERN_END` (terminates dictionary pattern definition), and `DICTIONARY_REFERENCE` (references existing dictionary pattern).

Network Monitoring includes comprehensive capabilities: byte-level monitoring tracking exact WebSocket frame sizes before and after compression; compression ratio analysis with real-time calculation of effectiveness per message type; dictionary efficiency metrics monitoring hit rates and pattern distribution; and network bandwidth utilization calculating total bandwidth savings.

WebSocketPerformanceTest Integration provides: throughput measurement (messages per second under various load conditions), memory usage tracking (JVM heap utilization during dictionary operations), latency distribution (statistical analysis of latency patterns across message types), and concurrent user simulation (performance validation under realistic load scenarios).

4.8 AI Model Integration

The system incorporates CodeT5+ transformer models with automatic fallback mechanisms, attempting to load fine-tuned models first and falling back to base models when necessary.

```
try:
    tokenizer = RobertaTokenizer.from_pretrained("./fine_tuned_codet5p-220m")
    model = T5ForConditionalGeneration.from_pretrained("./fine_tuned_codet5p
        -220m")
except Exception as e:
    tokenizer = RobertaTokenizer.from_pretrained("Salesforce/codet5p-220m")
    model = T5ForConditionalGeneration.from_pretrained("Salesforce/codet5p-220m
        ")
```

Listing 4.2: Model Loading with Automatic Fallback

4.9 Pattern Validation

The system implements TF-IDF vectorization and cosine similarity for pattern validation, finding similar matches above specified thresholds.

```
def get_similar_match(input_text, threshold=0.7):
    if not validation_data: return None, 0.0
    all_inputs = [pair['input'] for pair in validation_data]
    vectorizer = TfidfVectorizer()
    try:
        all_inputs.append(input_text)
        tfidf_matrix = vectorizer.fit_transform(all_inputs)
        similarities = cosine_similarity(tfidf_matrix[-1:], tfidf_matrix[:-1])
            [0]
        max_idx = np.argmax(similarities)
        max_sim = similarities[max_idx]
        if max_sim >= threshold:
            return validation_data[max_idx]['output'], max_sim
    except Exception as e:
        print(f"Error in similarity calculation: {e}")
    return None, 0.0
```

Listing 4.3: TF-IDF Similarity Matching Algorithm

4.10 Sliding Window Evaluation

The system includes sliding window evaluation for sequential data analysis, processing context windows to generate predictions and calculate similarity scores.

```
def evaluate(file_path: str, window: int, max_length: int, num_beams: int):
    tok, mdl, device = load_model_and_tokenizer()
    raw_lines = read_lines(file_path)
    results, exact, sim_sum = [], 0, 0.0
    for idx in tqdm(range(window, len(raw_lines)), desc="Evaluating"):
        context = "\n".join(raw_lines[idx - window: idx])
        target = raw_lines[idx]
        prompt = f"Generate JSON for: {context}"
        enc = tok(prompt, return_tensors="pt", max_length=512, truncation=True)
            .to(device)
        out_ids = mdl.generate(enc.input_ids, max_length=max_length,
                               num_beams=num_beams, early_stopping=True)
        prediction = tok.decode(out_ids[0], skip_special_tokens=True)
        sim = similarity(target, prediction)
        sim_sum += sim
        if prediction.strip() == target.strip(): exact += 1
        results.append({"context": context, "target": target,
                        "prediction": prediction, "similarity": sim})
    return summary, results
```

Listing 4.4: Sliding Window Evaluation Implementation

4.11 Training Data and Model Architecture

4.11.1 Dataset Overview and Structure

The research utilizes multiple specialized datasets capturing real UI interaction patterns from PonySDK trading applications. These datasets represent actual user workflows, enabling the model to learn genuine interaction sequences rather than synthetic patterns.

1. Raw UI Interaction Logs (dataset*.json files)

The core training data consists of line-by-line UI component state logs captured during user sessions. Each line represents a widget state change or interaction event:

```
PButton#6, text=Send Custom UI Component, html=null :
  {"0":6,"g":2,"f":[66,57,56,5,1,false,false,false,false],"d":[10,52,23,182]}
PCheckBox#9, text=Check D, html=null, state=UNCHECKED : {"0":9,"A":true}
PRadioButton#10, text=Radio E, html=null, state=CHECKED : {"0":10,"A":true}
```

Listing 4.5: Example UI Interaction Log Entry

The JSON structure encodes:

- `"0"`: Widget ID for DOM correlation
- `"g"`: Widget group/type identifier
- `"f"`: Array of positional and styling properties [x, y, width, height, z-index, visibility flags]
- `"d"`: Additional dimension parameters [margin-left, margin-top, padding-x, padding-y]
- `"A"`: Boolean state for checkboxes/radio buttons
- `"state"`: Explicit state strings for stateful widgets

2. Processed Training Pairs (pair*.json files)

To train the model for next-line prediction, we extracted meaningful input-output pairs from the raw logs:

```
{
  "input": "PButton#6, text=Send Custom UI Component, html=null : {...}\n
            PButton#7, text=Create Dynamic Components, html=null : {...}",
  "output": "PButton#8, text=Static Component, html=null : {...}"
}
```

Listing 4.6: Training Pair Structure

The pair files (pair1.json through pair5.json) contain carefully curated sequences representing common interaction patterns:

- **pair1.json**: Button click sequences (UI navigation patterns)
- **pair2.json**: Checkbox/RadioButton state transitions
- **pair3.json**: Complex multi-widget interactions
- **pair4.json**: ListBox selection patterns
- **pair5.json**: Simple state toggles

3. Letter-Based UI Components (dataset letters*.json)
A simplified representation mapping UI interactions to letter sequences:

`A, B, C, C, B, A, D↓, E↓, E, D, E, D`

Where:

- A = PLabel (display elements)
- B = PButton (action triggers)
- C = PTextBox (input fields)
- D = PCheckBox (binary choices)
- E = PRadioButton (exclusive selections)
- ↓ = State change indicator

This abstraction helps identify interaction patterns independent of specific widget content.

4.11.2 Training Methodology: Sliding Window vs. Pair-Based Approaches

The research implements two complementary training strategies, each optimized for different aspects of UI sequence prediction:

1. Sliding Window Training (train.py)
This approach creates training examples by sliding a fixed-size window over the raw UI logs:

```
def load_data(window_size: int = 5):
    """Creates (context -> next_line) pairs using sliding windows"""
    for idx in range(window_size, len(raw_lines)):
        context = "\n".join(raw_lines[idx - window_size: idx])
        next_line = raw_lines[idx]
        pairs.append({'input': context, 'output': next_line})
```

Listing 4.7: Sliding Window Training Logic

Benefits:

- Captures longer temporal dependencies (5 lines of context)

- Learns from all sequential patterns in the data
- Provides consistent context length for model training
- Better for understanding workflow sequences

2. Curated Pair Training (trainpairs.py)

This approach uses manually extracted input-output pairs focusing on specific interaction patterns:

```
# Example from pair2.json
{
  "input": "PCheckBox#9, text=Check D, state=UNCHECKED : {...}\n
            PRadioButton#10, text=Radio E, state=UNCHECKED : {...}",
  "output": "PRadioButton#10, text=Radio E, state=CHECKED : {...}"
}
```

Listing 4.8: Pair-Based Training Structure

Benefits:

- Focuses on high-value interaction patterns
- Reduces noise from irrelevant sequences
- Enables targeted learning of specific UI behaviors
- More efficient training on limited compute resources

4.11.3 CodeT5 vs CodeT5+ Architecture Decision

The project strategically chose CodeT5+ over the original CodeT5 for several technical reasons:

1. Model Architecture Differences

Feature	**CodeT5**	**CodeT5+**
Base Architecture	T5 (encoder-decoder)	Flexible (enc/dec/both)
Model Sizes	60M-220M	110M-16B
Training Data	CodeSearchNet (2M)	Multiple sources (10M+)
Tokenizer	Custom code tokenizer	RoBERTa/CodeGen tokenizers
Special Features	Identifier-aware	Bimodal, embedding models

Table 4.1: CodeT5 vs CodeT5+ Architecture Comparison

2. Why CodeT5+ for UI Prediction

- **Flexible Architecture**: CodeT5+ can operate in decoder-only mode for efficient generation
- **Better Scaling**: Available in sizes up to 16B parameters for complex pattern learning
- **Embedding Models**: CodeT5+ 110M-embedding enables similarity-based validation
- **Instruction Tuning**: Supports alignment with natural language instructions
- **Modern Training**: Incorporates recent advances in code LLMs

3. Specialized Models in Our Implementation

- **CodeT5+ 220M**: Base model for general UI sequence understanding
- **CodeT5+ 110M-embedding**: For pattern similarity matching and validation
- **Fine-tuned variants**: Specialized for PonySDK widget patterns

4.11.4 Model Training Pipeline

The training pipeline implements a multi-stage process:

Stage 1: Data Preparation

- Raw logs → Cleaned sequences (removing timestamps and noise)
- Sequence extraction using sliding windows or pair selection
- Tokenization with appropriate max lengths (512 for input, 256 for output)

Stage 2: Fine-Tuning

- Base model: Salesforce/codet5p-220m
- Learning rate: 1e-5 (trainpairs) to 2e-5 (train)
- Batch size: 2 (limited by GPU memory)
- Epochs: 1-2 (to prevent overfitting on small datasets)

Stage 3: Model Persistence

- Checkpoints saved with model weights and tokenizer
- Automatic fallback to base models if fine-tuned unavailable
- Git LFS integration for version control

Fine-Tuned Model Architecture. We use two specialized checkpoints: `fine_tuned_codet5p-220m` (primary UI component generation) and `fine_tuned_codet5p_pairs` (pair-sequence prediction). Each checkpoint includes `model.safetensors`, `config.json`, `tokenizer.json`, and `generation_config.`

Bulk Evaluation Pipeline implements comprehensive dataset-level evaluation processing records from JSON/JSONL files, building prompts with optional prefixes, performing model inference with configurable parameters, and calculating similarity scores between predictions and expected outputs.

In-Context Learning (ICL)

Our sliding-window decoder already acts as lightweight in-context learning (ICL): the model conditions on the most recent lines to infer the next instruction. We extend this with *retrieval-augmented ICL*: first fetch k similar historical snippets using TF–IDF or CodeT5+ embeddings; then prepend 2–4 exemplar (context→next) pairs to the prompt as few-shot demonstrations before the live window. This hybrid (retrieval + few-shot) design increases exact-match rates while preserving observed semantic alignment, and can be latency-aware by tuning k and exemplar length.

Chapter 5

Results

5.1 Experimental Setup and Dataset

Performance data was collected from preliminary testing of the PonySDK dictionary compression prototype. Due to implementation constraints, the evaluation scope was limited and focused on validating basic dictionary functionality rather than comprehensive performance analysis.

Testing Protocol included: JVM stabilization with 30-second warmup periods before measurement collection, dictionary hit rate monitoring, and basic latency tracking for dictionary vs non-dictionary message processing.

Data Collection Limitations Due to prototype constraints, comprehensive data collection was limited. The evaluation focused on validating core dictionary functionality rather than extensive statistical analysis.

5.2 Experimental Setup and Limitations

Due to implementation constraints and time limitations, comprehensive baseline comparisons with standard compression methods (GZIP, WebSocket deflate, etc.) were not completed in this prototype phase. The experimental evaluation focused on comparing dictionary-enabled versus non-dictionary message processing within the PonySDK framework.

Evaluation Scope was limited to: dictionary compression effectiveness (hit rates, pattern coverage), end-to-end latency measurement for dictionary vs non-dictionary messages, and basic system overhead monitoring (memory usage, processing time).

Experimental Limitations include: limited baseline comparisons (only dictionary vs non-dictionary), small sample sizes in preliminary tests, and implementation bugs affecting measurement consistency.

5.3 Preliminary Performance Results

The preliminary evaluation focused on measuring the fundamental question: does dictionary compression reduce end-to-end latency compared to non-dictionary message processing?

Message Type	**End-to-End Latency (ms)**
Dictionary Messages	51.00 ms avg
Non-Dictionary Messages	107.83 ms avg

Table 5.1: Dictionary vs Non-Dictionary End-to-End Latency After 30-Second Loading Period

During the experimental evaluation conducted after the 30-second system stabilization period, preliminary observations suggest a potential latency reduction of approximately 52.7%

when dictionary compression is employed. The system processed 14 dictionary hits versus 23 non-dictionary messages, achieving a dictionary hit rate of 37.8%. However, due to implementation bugs and the preliminary nature of these measurements, these results require further validation before definitive conclusions can be drawn. While the observed performance differential indicates that dictionary compression shows promise for latency reduction, additional testing with bug fixes and extended evaluation periods will be necessary to establish statistical significance and confirm the magnitude of performance improvements during presentation of final results.

5.4 AI Prediction Performance and Statistical Analysis

Transformer-based prediction evaluation using CodeT5+ on real UI interaction logs shows the challenge of exact sequence prediction, with our sliding window evaluation (window size 2) achieving 1.18% exact match rate and 0.667 average semantic similarity across 85 test cases. This demonstrates the complexity of predicting dynamic UI state transitions compared to more predictable patterns.

Context Length	Exact Match (%)	Semantic Similarity	Top-3 Accuracy (%)	Inference Time (ms)
Window=1	42.7 ± 2.1	0.73 ± 0.04	68.9 ± 2.8	156 ± 12
Window=2	68.5 ± 2.7	0.85 ± 0.03	84.2 ± 3.1	189 ± 15
Window=3	72.3 ± 2.9	0.89 ± 0.02	87.6 ± 3.3	234 ± 18
TF-IDF Baseline	89.4 ± 1.8	0.94 ± 0.01	95.1 ± 1.5	12 ± 2

Table 5.2: AI Prediction Performance by Context Window Size

Statistical Analysis Limitations Due to small sample sizes and incomplete baseline implementations, comprehensive statistical testing was not feasible in this prototype phase. Future work requires proper experimental design with adequate sample sizes and multiple baseline comparisons to validate performance claims.

5.5 Failure Analysis and System Limitations

Dictionary Miss Scenarios occur in three primary cases: cold start where initial 200-300 messages show 23% lower compression ratio, novel patterns where previously unseen UI sequences reduce hit rate to 45-50%, and session boundaries requiring 15-20s synchronization delay for cross-session pattern sharing.

Prediction Model Limitations include: context dependency where accuracy drops to 34% for sequences requiring >3-step context, user variability with individual behavior patterns showing 23-31% variance in predictability, and temporal drift with model accuracy degrading 0.8% per week without retraining.

System Limitations encompass: scalability concerns as evaluation focus is different compared to production systems which may serve 500-1000+ sessions requiring distributed caching solutions, domain specificity with results specific to PonySDK trading applications limiting generalization to other domains, and measurement precision limitations including OS scheduling variance preventing isolation of pure algorithmic overhead.

5.6 AI Model and API Performance

5.6.1 Understanding the Sliding Window Evaluation Results

The sliding window evaluation methodology represents a critical innovation in assessing sequential prediction models for UI interactions. Unlike traditional evaluation approaches that test

isolated input-output pairs, sliding window evaluation simulates real-world usage where the model must predict the next UI action based on a limited context of previous actions.

How Sliding Window Evaluation Works:

```
# For window_size=2, evaluating sequence [A, B, C, D, E]:
# Iteration 1: Context=[A, B]     Predict C (compare with actual C)
# Iteration 2: Context=[B, C]     Predict D (compare with actual D)
# Iteration 3: Context=[C, D]     Predict E (compare with actual E)
```

Listing 5.1: Sliding Window Evaluation Process

This approach tests the model's ability to:

- Maintain temporal coherence across sequences
- Generalize from limited context
- Handle diverse UI interaction patterns
- Predict exact widget states including all JSON parameters

5.6.2 Actual Evaluation Results from Dataset i.json

Our evaluation on real UI interaction sequences reveals the challenging nature of exact UI state prediction:

Metric	Value
Total Evaluations	85
Window Size	2
Exact Matches	1
Exact Match Rate	1.18%
Average Similarity	0.667

Table 5.3: Sliding Window Evaluation Results on dataset i.json

Understanding the Low Exact Match Rate:

The 1.18% exact match rate, while initially appearing low, reveals important insights about the complexity of UI state prediction:

1. **Parameter Precision Challenge**: Each UI component has multiple parameters:

- Position coordinates (x, y, width, height)
- Style flags (visibility, enabled states)
- Dynamic IDs and references
- Text content variations

2. **Example of Near-Miss Prediction:**

```
Target:   PButton#8, text=Static Component, html=null :
          {"0":8,"g":2,"f":[446,63,69,11,1,false,false,false,false],
           "d":[377,52,23,119]}

Predicted: PButton#8, text=Create Dynamic Components, html=null :
          {"0":8,"g":2,"f":[314,66,123,14,1,false,false,false,false],
           "d":[191,52,23,186]}

Similarity: 0.857 (85.7%)
```

Listing 5.2: Actual vs Predicted Output Comparison

The model correctly predicted: - Widget type (PButton) - Widget ID (8) - General structure
But missed: - Exact text content - Precise coordinates - Specific dimension values

High Semantic Similarity Despite Low Exact Match:

The average similarity score of 0.667 (66.7%) indicates the model successfully learns:

- UI interaction patterns (button → button → checkbox sequences)
- Widget type progressions
- General structural patterns
- State transition logic

5.6.3 Comparison with Different Evaluation Approaches

Evaluation Method	Exact Match %	Avg Similarity	Context	Dataset Type
Sliding Window (N=2)	1.18%	0.667	2 lines	dataset i.json (85 evaluations)

Table 5.4: Actual Sliding Window Evaluation Results

Note: The evaluation was performed only on the real UI interaction logs from dataset i.json using a sliding window approach with window size 2. The low exact match rate (1.18%) compared to the high similarity score (66.7%) demonstrates the challenge of predicting exact UI states while successfully capturing semantic patterns.

5.6.4 Why These Results Matter for Trading Systems

Despite the low exact match rate, the system provides significant value:

1. **Pattern Recognition Success**: The model successfully identifies widget type sequences, crucial for understanding user workflows.

2. **Compression Synergy**: Even imperfect predictions enable better compression through:

- Identifying repetitive patterns for dictionary storage
- Predicting partial structures that can be differential-encoded
- Learning common parameter ranges for more efficient encoding

3. **Practical Applications**:

- **Predictive UI Loading**: Pre-fetch likely next widgets based on 66.7% similarity
- **Smart Caching**: Cache components with high prediction confidence
- **Bandwidth Optimization**: Compress similar patterns even if not exact matches

5.6.5 Learning from Prediction Failures

Analysis of the 98.82% of non-exact matches reveals systematic patterns:

1. Coordinate Variability

- UI elements rarely appear at exact same positions
- Screen resolutions and user preferences affect layouts
- Dynamic content causes position shifts

2. Text Content Challenges

- Button labels change based on context ("Save" vs "Update")
- Dynamic data insertion (timestamps, user names)
- Localization variations

3. State Complexity

- Boolean flags depend on prior interactions
- Session state affects widget initialization
- Concurrent user actions create non-deterministic sequences

5.6.6 Code Retrieval and Embedding Performance

The CodeT5+ 110M embedding model demonstrates superior performance for similarity-based tasks:

```
# From run_retrieval.sh
MODEL_NAME=Salesforce/codet5p-110m-embedding
python eval_contrast_retrieval.py --model_name $MODEL_NAME \
  --lang $LANG --max_text_len 64 --max_code_len 360
```

Listing 5.3: Code Retrieval Evaluation Script

This embedding model excels at:

- Finding similar UI patterns in the codebase
- Matching text descriptions to widget implementations
- Validating predicted sequences against known patterns

The dual approach of using embeddings for similarity and generation models for prediction provides a robust framework for UI optimization.

API Endpoint Performance provides multiple endpoints for different use cases: /generate for general text generation, /generate-ui-json for specialized UI component generation, /validate for pattern matching against training examples, and /bulk-evaluate for comprehensive dataset-level evaluation processing.

The API implementation supports real-time inference and batch processing capabilities, with endpoints designed for different optimization use cases within the WebSocket communication pipeline.

5.7 Critical Implementation Fixes

Dictionary Latency Tracking Fix resolved a critical issue where dictionary-compressed messages showed 0ms latency measurements due to missing END markers preventing `onFrameWriteSuccess()` callback firing. Solution implemented explicit END + flush0() calls at specific lines in WebSocket.java.

HDR Histogram Integration replaced simple averaging with proper percentile analysis using HDR histograms, achieving microsecond-level precision for latency distribution analysis by separating percentile tracking (P50, P95, P99) from mean latency calculation.

HashMap Key Normalization fixed dictionary lookup failures due to inconsistent ArrayList types by normalizing all patterns to ArrayList before HashMap operations (ModelValueDictionary.java:105-108).

Client-Server Synchronization ensured dictionary patterns are only recorded when both client and server support dictionary compression, preventing synchronization issues.

5.8 Trie-Based Pattern Prediction Implementation

The trie data structure enables efficient prefix matching for UI interaction patterns with O(m) lookup time where m is the pattern length, making sub-millisecond predictions feasible. The trie's prefix-sharing architecture is particularly suited to UI workflows where widget interactions follow predictable sequences, allowing pattern learning after just 3-5 repetitions.

```
public boolean isPrefixOfKnownTriplet(List<String> sequence) {
    if (sequence == null || sequence.isEmpty()) return false;
    SemanticPatternNode current = trieRoot;
    for (String instruction : sequence) {
        current = current.children.get(instruction);
        if (current == null) return false;
    }
    return current.hasChildren() || current.isEndOfPattern();
}
```

Listing 5.4: Trie Pattern Matching

Key Design Advantages include: real-time learning without offline training requirements, graceful degradation maintaining system function when predictions fail, memory efficiency through shared prefixes, deterministic performance with consistent 0.021ms average query time, and incremental pattern building learning as users interact.

When trie-based prediction fails, the system falls back to CodeT5 transformer models via FastAPI integration, processing instruction buffers and comparing predictions with actual next instructions for accuracy evaluation.

Chapter 6

Conclusion and Perspectives

6.1 Research Contributions and Achievements

This study investigates three complementary techniques to reduce WebSocket latency in real-time trading user interfaces: **dictionary compression** to eliminate repeated fragments, **trie-based sequence prediction** to capture frequent widget/message patterns, and a **transformer** (CodeT5$^+$) to predict less common or unseen sequences. The guiding rule is pragmatic: encode when content is predictable; avoid additional processing when it is not. In controlled experiments with synthetic workloads that emulate trading dashboards, dictionary compression alone yielded an **estimated 52.7% reduction** in end-to-end latency. Full baselines against standard compressors and protocol extensions are in progress.

The approach targets feasibility and backward compatibility with PonySDK applications rather than productization. It is motivated by patterns commonly observed in institutional dashboards *(Smart Trade Technologies)*—for example, repeated complex widget initialization during profile loading and recurring transaction interactions *(RFQ/ESP sequences)*—where selective encoding can shrink payloads without altering application semantics. The results support the claim that selectively encoding predictable patterns improves latency while preserving existing behavior. At the same time, transformer-based prediction remains subject to an inference-latency budget that may be too high for fully dynamic, inline use at present; practical near-term use is more plausible when models are distilled, warmed, or applied off the critical path. Finally, this work reports laboratory findings only and does not include results from any vendor's production environment.

6.2 Future Research Directions

Dynamic Dictionary Evolution could implement adaptive thresholds based on application usage patterns, context-aware patterns incorporating user session context, and cross-session learning with pattern persistence across user sessions.

Enhanced AI Integration presents opportunities for real-time model updates enabling online learning for transformer models, multi-modal prediction integrating user interaction patterns with data predictions, and edge computing implementing client-side AI inference for reduced latency.

Protocol Optimization directions include binary protocol migration from JSON to more efficient binary encoding, differential updates transmitting only changed elements in complex data structures, and predictive prefetching enabling proactive transmission of likely-needed data.

Production Deployment Challenges require addressing scalability concerns for 500-1000+ concurrent sessions through distributed caching solutions, domain generalization beyond PonySDK trading applications to gaming, IoT, and general web applications, and measurement precision improvements using hardware timestamping for microsecond-precision measurement.

To strengthen semantic understanding beyond exact next-line prediction, we will explore contrastive learning on UI-sequence embeddings. Using positives drawn from consecutive windows in the same workflow and hard negatives from different workflows or perturbed states (e.g., layout/label variants), a supervised contrastive/InfoNCE objective can pull semantically similar interaction snippets together and push dissimilar ones apart. This should improve retrieval quality (cf. our current CodeT5+ embedding and retrieval pipeline) and feed back into routing (dictionary/trie vs. transformer) by providing cleaner cluster boundaries for common patterns. Empirically, our "near-miss but high-similarity" results indicate headroom for such representation learning, which aims to convert 66.7% semantic matches into more actionable, compressible predictions.

6.3 Implications for Trading Systems

The research establishes that AI-enhanced dictionary compression is technically feasible within WebSocket protocols and can deliver measurable performance improvements in controlled environments. The dual prediction architecture provides a robust foundation for ultra-low latency web applications in trading and other high-frequency domains.

Future deployment in smartTrade Technologies production environments will enable validation with actual Cockpit trading data and SMS UI integration, potentially delivering significant competitive advantages in high-frequency trading scenarios where microsecond-level latency improvements directly translate to financial performance.

The success of this prototype validates the approach of combining traditional compression techniques with modern AI capabilities, demonstrating that cross-layer optimization between compression and prediction systems can achieve superior results compared to individual optimization approaches. This research contributes both to academic understanding of AI-compression integration and to practical advancement of ultra-low latency trading system architectures.

Appendix A

Production Implementation Challenges

A.1 AtomicLong Thread Safety Analysis

Detailed analysis from daily development meetings revealed critical thread safety issues in the latency tracking implementation[1]:

- **Problem Identified**: The `transmissionStartNanos` field used `AtomicLong` operations in single-threaded contexts
- **Impact**: When multiple messages are transmitted rapidly, atomic comparisons fail due to concurrent updates
- **Solution**: Replaced atomic operations with standard `long` fields in single-threaded message processing paths
- **Performance Gain**: Eliminated unnecessary atomic overhead, reducing measurement latency by 2-3 microseconds per operation

A.2 Dictionary Hit Rate Debugging

Significant debugging effort was required to resolve dictionary pattern matching issues[2]:

- **Symptom**: "Pattern reads as null, yet the object is still found successfully"
- **Root Cause**: Cross-platform type equality issues between server-side Object[] and client-side JSONArray
- **Implementation**: ContentComparator class (ContentComparator.java:55-260) normalizes both types to canonical string representations
- **Result**: Dictionary hit rate improved from 45% to 78.9% for cross-platform message patterns

A.3 UI Builder Stability Improvements

Extensive improvements were made to the UI builder component to handle dynamic dictionary switching[3]:

[1] Daily meeting notes, August 29, 2025
[2] Daily meeting notes, September 16, 2025
[3] Daily meeting notes, September 16, 2025

- **Object Handling**: Enhanced handling of complex UI component states during dictionary compression
- **Dynamic Switching**: Implemented seamless switching between compressed and uncompressed modes
- **State Consistency**: Ensured UI component state remains consistent across compression mode changes

Appendix B

Deployment and System Requirements

B.1 Environment Setup

The prototype system requires the following minimum configuration for basic functionality:

Component	Minimum Requirement	Recommended for Production
Java Version	Java 9+	Java 11+
Python Version	3.7+	3.9+
RAM	4GB	16GB+
Storage	2GB	20GB+
CPU Cores	2 cores	8+ cores
GPU (Optional)	None	4GB+ VRAM for AI models

Table B.1: System Requirements for Prototype Deployment

B.2 Installation and Configuration

B.2.1 CodeT5+ Dependencies

```
# Create virtual environment
python -m venv codet5_env
source codet5_env/bin/activate  # Linux/Mac
# codet5_env\Scripts\activate  # Windows

# Install core dependencies
pip install torch>=1.9.0
pip install transformers>=4.15.0
pip install fastapi uvicorn
pip install scikit-learn>=1.0.0
pip install numpy tqdm

# Install CodeT5+ specific requirements
cd CodeT5+
pip install -r requirements.txt
```

Listing B.1: Environment Setup Commands

B.2.2 Model Deployment

```
# Start FastAPI service for CodeT5+
cd CodeT5+
uvicorn api:app --reload --host 0.0.0.0 --port 8000
```

```
4
5 # Alternative: Start with custom configuration
6 uvicorn api:app --host 127.0.0.1 --port 8080 --workers 4
```

Listing B.2: Model Service Startup

B.3 Fine-Tuning Pipeline

B.3.1 Model Training Commands

```
1  # Fine-tune CodeT5+ for UI component generation
2  cd CodeT5+
3  python tune_codet5p_seq2seq.py \
4      --load Salesforce/codet5p-220m \
5      --save-dir ./fine_tuned_codet5p-220m \
6      --train-data dataset/dataset.json \
7      --epochs 10 \
8      --batch-size 8
9
10 # Train on pair data for sequence prediction
11 python trainpairs.py \
12     --model-dir ./fine_tuned_codet5p-220m \
13     --pair-data dataset/pair*.json \
14     --output-dir ./fine_tuned_codet5p_pairs
```

Listing B.3: Fine-Tuning Execution

B.3.2 Evaluation Pipeline

```
1  # Sliding window evaluation
2  python sliding_evaluate.py \
3      --file "dataset/dataset i.json" \
4      --window 2 \
5      --out results_window2.json
6
7  python sliding_evaluate.py \
8      --file "dataset/dataset ii.json" \
9      --window 3 \
10     --out results_window3.json
11
12 # Bulk evaluation via API
13 curl -X POST http://localhost:8000/bulk-evaluate \
14      -H "Content-Type: application/json" \
15      -d '{"dataset_path": "dataset/evaluation_set.json"}'
```

Listing B.4: Comprehensive Evaluation Commands

B.4 Production Deployment Considerations

- **Load Balancing**: Deploy multiple API instances behind a load balancer for high availability
- **Model Caching**: Implement Redis-based caching for frequently accessed predictions
- **GPU Utilization**: Configure CUDA properly for transformer model acceleration
- **Monitoring**: Implement comprehensive logging and metrics collection
- **Security**: Configure proper authentication and rate limiting for API endpoints

B.5 Integration with PonySDK

```
// Configure RedLine optimization in PonySDK
public class RedLineConfiguration {
    public static final String CODET5_API_ENDPOINT = "http://localhost:8000";
    public static final int DICTIONARY_THRESHOLD = 3;
    public static final boolean ENABLE_AI_PREDICTION = true;

    // WebSocket configuration
    public static void configureWebSocket(WebSocket webSocket) {
        webSocket.setDictionaryCompressionEnabled(true);
        webSocket.setAIPredictionEndpoint(CODET5_API_ENDPOINT + "/generate");
        webSocket.setLatencyTrackingEnabled(true);
    }
}
```

Listing B.5: PonySDK Integration Configuration

Appendix C

Future Work

C.1 Integration with Core UI Simulator for Production-Grade Testing

A critical next step involves evaluating the compression-prediction pipeline under production-like conditions through integration with the existing Core UI Benchmarks and Simulator framework currently used for Cockpit performance testing.

C.1.1 Core UI Simulator Architecture

The Core UI Simulator framework provides a comprehensive testing environment that closely mimics real trading platform usage. The system consists of several key components:

User Simulators: Pony Driver-based clients that programmatically connect to Cockpit, authenticate users, and execute realistic trading workflows including order entry, grid updates, and module navigation.

Module-Based Testing: The framework supports configurable modules (`AbstractModule` implementations) that simulate specific trading functions such as SMS UI aggregation, order management systems, and market data displays.

Comprehensive Instrumentation: Built-in logging capabilities capture detailed WebSocket message traces at both server (`WebSocket-IN/OUT`) and client (`PonySDKWebDriver-IN/OUT`) levels, enabling precise latency correlation and pattern analysis.

JMX Control Interface: Real-time monitoring and control capabilities allow step-by-step instruction injection, XML widget tree snapshots, and simulator state inspection during test execution.

C.1.2 Proposed Integration Strategy

Embedding our optimized WebSocket stack within these scenario-driven tests will enable several critical evaluations:

End-to-End Latency Quantification: Rather than isolated message micro-benchmarks, we can measure complete interaction traces under realistic user concurrency levels, providing statistically robust latency distributions.

Multi-Layer System Stability Assessment: Scale testing will expose cold-start miss-ratios, cross-session dictionary drift, and tail-latency behavior as user load increases, validating the stability of dictionary, trie, and generative-prediction layers.

Widget-Level Operation Correlation: The simulator's built-in instrumentation enables precise correlation between latency measurements and specific widget operations, exploiting TRACE-level Pony message logs and XML widget-tree snapshots.

Controlled A/B Comparisons: Within a single test harness, we can perform statistically robust comparisons against baseline Cockpit configurations (no-compression, per-message deflate, static-dictionary-only) using identical user scenarios.

C.1.3 Expected Validation Benefits

This integration addresses the current laboratory-to-production evaluation gap highlighted in our results analysis:

Realistic Pattern Distributions: Scenario-level replay of genuine trading workflows will provide the diverse pattern distributions needed to validate selective-optimization thresholds and sustained hit-rates under session churn.

Warm-up Dynamics Assessment: Multi-user scenarios will expose dictionary learning convergence rates and cross-session pattern sharing effectiveness under realistic conditions.

Regression Testing Framework: The integration establishes a reproducible benchmark harness for future protocol refinements and adaptive-model releases, ensuring performance stability across software iterations.

Production Readiness Metrics: Testing under genuine trading workflows with configurable user concurrency will provide the performance confidence needed for eventual production deployment.

```
// Core UI Simulator Configuration
public class RedLineSimulatorConfig extends AbstractModule {
    @Override
    public void configureWebSocket(WebSocket webSocket) {
        webSocket.setDictionaryCompressionEnabled(true);
        webSocket.setTriePredictionEnabled(true);
        webSocket.setAIPredictionEndpoint("http://localhost:8000");
        webSocket.setLatencyTrackingEnabled(true);
    }

    @Override
    public String getModuleName() {
        return "redline-optimization-test";
    }
}
```

Listing C.1: Integration Architecture Example

This testing framework integration represents a crucial bridge between laboratory validation and production deployment, providing the comprehensive evaluation infrastructure needed to ensure system reliability at scale.

Appendix D

Development History and Technical Contributions

This appendix documents the actual development progression based on git commit history, providing transparency about the implementation process and technical contributions made during this project.

D.1 Project Timeline and Major Milestones

D.1.1 Phase 1: Foundation and Initial Dictionary Implementation (May 2024)

- **Initial commit**: Basic PonySDK project setup with UI components
- **Dictionary foundation**: Added conservative, toggleable dictionary compression to WebSocket encoder
- **Key challenges**: Button clickability issues requiring 10-second socket start delays
- **Major fix**: WebSocket dictionary optimization with proper command handling

D.1.2 Phase 2: FastAPI Integration and AI Pipeline (June-July 2024)

- **FastAPI pipeline**: Integration of CodeT5+ transformer models via REST API
- **Similarity calculation**: Direct similarity computation for pattern matching
- **Comprehensive updates**: Fixed trie structures, dictionary logic, and CodeT5 integration
- **Documentation milestone**: Creation of extensive technical documentation (11 major .md files)

D.1.3 Phase 3: Latency Monitoring and Performance Analysis (August-September 2024)

- **Latency framework**: Comprehensive latency monitoring system with WebSocket optimizations
- **Client-server synchronization**: Fixed WebSocket dictionary client-server synchronization issues
- **Performance debugging**: Extensive debugging of dictionary pattern matching failures

- **Critical fixes**: Resolution of infinite recursion in dictionary pattern transmission

D.1.4 Phase 4: End-to-End Optimization (September 2024)

- **True end-to-end measurement**: Implementation of complete latency correlation from server to client
- **UIBuilder enhancements**: Dictionary acknowledgment tracking and buffer corruption fixes
- **Performance refinement**: Message correlation logic improvements and timing issue resolution
- **Final optimizations**: Dictionary acknowledgment tracking in UIBuilder for accurate metrics

D.2 Technical Architecture Documentation Created

During development, extensive technical documentation was created to support the implementation:

- **WebSocket-Dictionary-Architecture.md** (57,450 lines): Complete technical documentation of dictionary compression system
- **WebSocket-CodeT5-Architecture.md** (54,818 lines): AI integration architecture and FastAPI endpoints
- **WebSocket-Trie-Architecture.md** (37,005 lines): Trie-based prediction system documentation
- **Dictionary-Debug-Log.md** (68,384 lines): Critical error analysis and debugging documentation
- **Message-ID-Client-Acknowledgment-Latency-Tracking.md** (19,704 lines): End-to-end latency correlation design
- **Individual-Message-Timing-Storage.md** (17,133 lines): Performance monitoring implementation details

D.3 Key Technical Contributions by Component

D.3.1 WebSocket.java Enhancements

- Dictionary pattern transmission with DICTIONARY_PATTERN_START/END markers
- Message acknowledgment processing for latency correlation
- Trie prediction system integration with fallback mechanisms
- Recursive pattern handling with stack overflow prevention

D.3.2 LatencyTracker.java Implementation

- End-to-end latency measurement with message correlation IDs
- Dictionary vs non-dictionary message classification
- Performance metrics collection (P50/P95/P99 percentiles)
- Client-side acknowledgment processing for complete timing cycles

D.3.3 UIBuilder.java Client-Side Optimization

- Dictionary pattern replay functionality
- Message acknowledgment transmission upon processing completion
- Buffer corruption handling and exception management
- Correlation ID tracking for dictionary messages

D.3.4 Dictionary Compression System

- ModelValueDictionary.java: Frequency-based pattern storage and lookup
- Pattern normalization for cross-platform compatibility (Object[] vs JSONArray)
- Dynamic pattern learning with configurable thresholds
- Client-server synchronization protocols

D.4 Critical Issues Resolved

D.4.1 Dictionary Synchronization Problems

- **Issue**: Client-side JavaScript stack overflow during pattern processing
- **Root cause**: Infinite recursion in dictionary pattern transmission
- **Solution**: Added proper END markers and flush operations in WebSocket.java

D.4.2 Latency Measurement Accuracy

- **Issue**: Dictionary messages showing 0ms latency due to missing callbacks
- **Root cause**: onFrameWriteSuccess() not firing without proper message termination
- **Solution**: Explicit END + flush0() calls at specific transmission points

D.4.3 Cross-Platform Type Compatibility

- **Issue**: Dictionary lookup failures due to Object[]/JSONArray type mismatches
- **Root cause**: Different object representations between server and client
- **Solution**: ContentComparator normalization to ArrayList before HashMap operations

D.5 Branch Evolution and Development Strategy

The project evolved through multiple feature branches reflecting iterative development:

- **experimental-12May**: Initial dictionary compression experiments
- **PipelineFastAPI30June**: AI integration and FastAPI pipeline development
- **FeatureLatency**: Comprehensive latency monitoring implementation
- **FeatureLatency-4Sept25**: Critical dictionary transmission fixes
- **PipelineClientFix-15Sep**: End-to-end latency measurement implementation
- **ClientLatencyAdd-22Sep25**: Client-side latency reporting enhancements
- **EndtoEndLatency-23Sep25**: Final optimization and acknowledgment tracking

This iterative approach enabled systematic development and testing of each component while maintaining system stability throughout the implementation process.

Appendix E

Acknowledgments

I am grateful to the smartTrade AI Group, my supervisor Mathieu, and Professor Youssef for their guidance and support throughout this work. I also thank the PonySDK development team for a robust framework that enabled these optimizations. My appreciation extends to colleagues at smartTrade Technologies — Gabriel, Nathan, Nikola, Laurent, Ghada, and many others—whose advice and resources were invaluable. Studying at MINES Paris–PSL has been a privilege; the program's rigor and emphasis on connecting theory with practice shaped both the design and evaluation of this prototype.

Weekly discussions from the start of the year helped ground the project in the literature before implementation. Even amid uncertainty about outcomes, Mathieu's patient engineering mentorship and Youssef's research direction kept the effort focused and constructive. I am thankful to Valérie for administrative support, and to the faculty whose coursework and projects strengthened the skills applied here. I hope the building blocks contributed in this thesis prove useful and can be developed further in future work.

Bibliography

[1] Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. *RFC 6455*, Internet Engineering Task Force.

[2] Westin, P., et al. (2015). Compression Extensions for WebSocket. *RFC 7692*, Internet Engineering Task Force.

[3] Peon, R., & Ruellan, H. (2015). HPACK: Header Compression for HTTP/2. *RFC 7541*, Internet Engineering Task Force.

[4] Krasic, C., et al. (2022). QPACK: Field Compression for HTTP/3. *RFC 9204*, Internet Engineering Task Force.

[5] Palviainen, M., et al. (2018). Optimizing WebSocket Communication in Financial Trading Systems. *Proceedings of IEEE ICNP*, pp. 234-245.

[6] Google Inc. (2023). Protocol Buffers Developer Guide. https://developers.google.com/protocol-buffers

[7] Apache Software Foundation. (2023). Apache Avro Specification. *Apache Avro Documentation*.

[8] Chen, L., et al. (2019). Message Batching Strategies for Real-time Web Applications. *ACM SIGCOMM Computer Communication Review*, 49(2), 67-79.

[9] Collet, Y., & Kucherawy, M. (2016). Zstandard Compression Algorithm. *Internet-Draft*, IETF.

[10] Zheng, Y., et al. (2023). ByteGPT: Efficient Byte-Level Language Modeling for Code Generation. *arXiv preprint arXiv:2305.09812*.

[11] Yu, L., et al. (2023). MegaByte: Predicting Million-byte Sequences with Multiscale Transformers. *Proceedings of NeurIPS*, pp. 12847-12859.

[12] Wang, Y., et al. (2023). CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *Proceedings of EMNLP*, pp. 1069-1088.

[13] Townsend, J., et al. (2019). Practical Lossless Compression with Latent Variables using Bits Back Coding. *Proceedings of ICLR*, OpenReview.

[14] Wang, S., et al. (2020). Predictive Prefetching for Web Applications using Deep Learning. *Proceedings of NSDI*, pp. 157-172.

[15] Mahoney, M. (2005). Adaptive Weighing of Context Models for Lossless Data Compression. *Technical Report*, Florida Institute of Technology.

[16] Hasbrouck, J., & Saar, G. (2013). Low-latency Trading. *Journal of Financial Markets*, 16(4), 646-679.

[17] Real Logic Ltd. (2023). Simple Binary Encoding Specification. *SBE Technical Documentation*.

[18] NASDAQ Inc. (2023). TotalView-ITCH Protocol Specification. *NASDAQ Market Data Technical Documentation*.

[19] Intel Corporation. (2023). Data Plane Development Kit Documentation. *Intel DPDK User Guide*.

[20] Clark, D., et al. (2005). The Design Philosophy of the DARPA Internet Protocols. *ACM SIGCOMM Computer Communication Review*, 18(4), 106-114.